T3 – Nicolas Sanchez – 10/30/2026

Repositorio: `https://github.com/nico031022/t3_image_nicolas_sanchez`

1. Eliminación de ruido periódico con filtros notch (p1)

Implementé una cadena de filtrado en frecuencia con dos familias de filtros notch, ideal y gaussiano, para sacar el patrón periódico de `P1.CR2`. En el espectro aparecen cinco componentes de ruido: tres del mosaico de Bayer y un par de un patrón que se repite cada 4 columnas. Al final elegí un notch gaussiano con $\sigma = 0.03$ ciclos/píxel y padding reflejo de 128 píxeles. Con esa configuración el patrón desaparece del cielo y de la nebulosa sin cambiar el promedio de la imagen, aunque quedan algunos artefactos en las estrellas saturadas más pequeñas.

Lectura de la imagen

`P1.CR2` es un RAW de Canon y OpenCV no lo puede leer, por eso usé `rawpy`. Tomé `raw_image_visible`, que es el mosaico de Bayer sin interpolar, de 2856×4290 píxeles, y lo traté como una imagen en escala de grises. La normalicé con el nivel de negro (promedio de los cuatro canales) y el nivel de blanco del sensor, sin recortar: los píxeles saturados quedan un poco sobre 1.

Implementación

La función `filtrar_notch` hace la cadena completa en seis pasos: (1) agrega el padding con `aplicar_padding`, (2) calcula `fft2` y `fftshift`, (3) construye H del mismo tamaño que la imagen con padding, (4) multiplica el espectro complejo por H, (5) vuelve con `ifftshift` e `ifft2` y se queda con la parte real, y (6) recorta al tamaño original con `recortar_padding`. El tipo de filtro, los centros, el ancho y el tipo y tamaño del padding son argumentos de la función, y en cada script quedan como constantes al inicio.

Trabajé las frecuencias en ciclos/píxel. `crear_grilla_frecuencias` arma las matrices U y V con `fftfreq` y `fftshift`, y en el mismo orden que el espectro centrado. Así los centros y los anchos no dependen del tamaño de la imagen, lo que es importante porque con padding el tamaño cambia. Para un centro (u_0, v_0), `distancia_a_centro` calcula

$$d_u = \min\big(|u - u_0|,\, 1 - |u - u_0|\big), \qquad d_v = \min\big(|v - v_0|,\, 1 - |v - v_0|\big), \qquad D = \sqrt{d_u^2 + d_v^2}. \tag{1.1}$$

El mínimo con $1 - |u - u_0|$ está porque el espectro es periódico con periodo 1. Sin eso, un notch centrado en -0.5 no taparía lo que queda al otro lado del borde, cerca de $+0.5$.

Con esa distancia armé los dos filtros. Si hay varios centros k, se multiplica el filtro de cada uno:

$$H_{\text{ideal}}(u,v) = \begin{cases} 0 & \text{si } D_k \le r \text{ para algún } k \\ 1 & \text{si no} \end{cases} \qquad H_{\text{gauss}}(u,v) = \prod_k \left(1 - e^{-D_k^2/2\sigma^2}\right). \tag{1.2}$$

El gaussiano es el pasa altos gaussiano de la clase 14 [4], pero con la distancia medida desde el centro del notch y no desde el origen.

En este caso solo hay que dar un centro de cada par: `agregar_simetricos` agrega $(-u_0, -v_0)$ automáticamente. Si cada coordenada del centro es 0 o ± 0.5, el simétrico es el mismo punto (esto pasa con los picos de Bayer) y no se agrega dos veces. Para comparar las dos familias con un mismo ancho usé el radio donde $H = 0.5$: en el ideal es r y en el gaussiano es $\sigma\sqrt{2\ln 2} \approx 1.18\,\sigma$.

Revisé que $H(0,0) = 1$ en los dos filtros, así que el promedio de la imagen no cambia, y que $\max |H(u,v) - H(-u,-v)| = 0$ en el ideal y $\approx 7 \times 10^{-15}$ en el gaussiano. Por eso la imagen filtrada sale real: la parte imaginaria después de la `ifft` queda en $\sim 10^{-15}$.

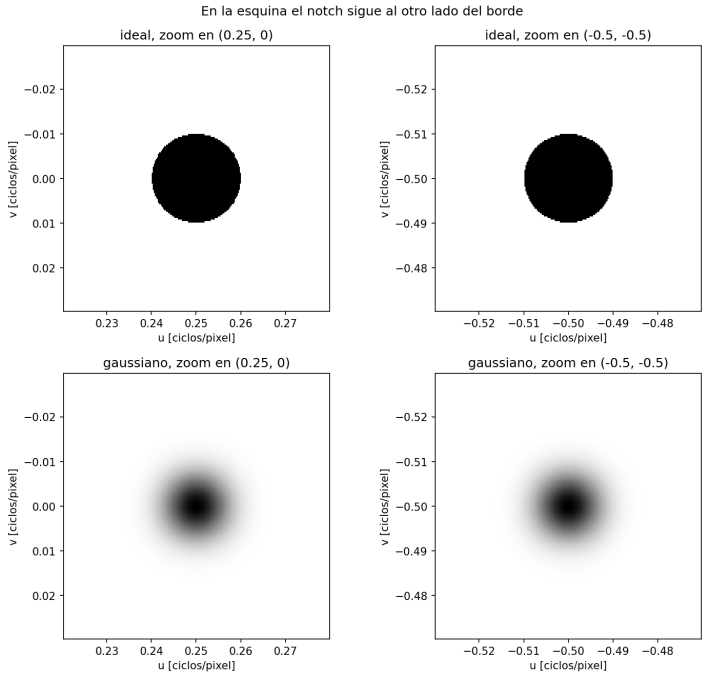

Figura 1.1: H cerca de $(0.25, 0)$ y de la esquina $(-0.5, -0.5)$. Arriba: ideal con $r = 0.01$. Abajo: gaussiano con $\sigma = 0.005$. Son anchos de prueba, solo para ver la forma. En la esquina el agujero sigue al otro lado del borde gracias a la distancia periódica.

1.1. Espectro y posición de los notches

En el espectro de la Figura 1.2 se ve el contenido de la imagen concentrado alrededor del centro y algunos puntos brillantes aislados: dos sobre el eje horizontal y otros en los bordes y esquinas.

Para no elegir los picos a ojo, busqué los máximos de $|F|$ fuera de un círculo de radio 0.05 alrededor del centro. En el perfil radial, desde ese radio hacia afuera el contenido de la imagen ya es bajo. Busqué 10 picos, el doble de los cinco que se veían, y después de encontrar cada uno borré una ventana de 7×7 a su alrededor (dando la vuelta en los bordes del espectro, si no el pico de la esquina aparecía dos veces).

k	u	v	$\lvert F\rvert$/mediana
1	-0.5000	-0.5000	1474.0
2	-0.5000	0	1254.8
3	0	-0.5000	1211.0
4	-0.2501	0	325.2
5	$+0.2501$	0	325.2
6–10	a 2–4 píxeles de los picos 1, 3, 4 y 5		≤ 57.0

Tabla 1.1: Picos encontrados con radio excluido 0.05. $|F|$ se divide por la mediana del espectro para que el fondo quede cerca de 1.

Entre el pico 5 (325) y el 6 (57) hay un salto claro, y los picos 6 a 10 están pegados a los anteriores: son parte del mismo pico y no componentes nuevas. Con radios excluidos de 0.02, 0.1 y 0.2 salen los mismos cinco picos, entonces el radio no es crítico.

Los picos 1, 2 y 3 tienen periodo 2 en filas, en columnas o en ambas. Corresponden al mosaico de Bayer [2]: como leí el RAW sin interpolar, cada píxel tiene el filtro de un color distinto y la intensidad alterna en un patrón de 2×2. Como el alto y el ancho son pares, -0.5 y $+0.5$ son la misma frecuencia y cada uno de estos picos es su propio simétrico. Los picos 4 y 5 son un par simétrico en $u = \pm 0.25$: un patrón que se repite cada 4 columnas. En $v = \pm 0.25$ no aparece nada, entonces este patrón es solo horizontal. En el zoom de la Figura 1.4 se ve que el pico de 0.25 se

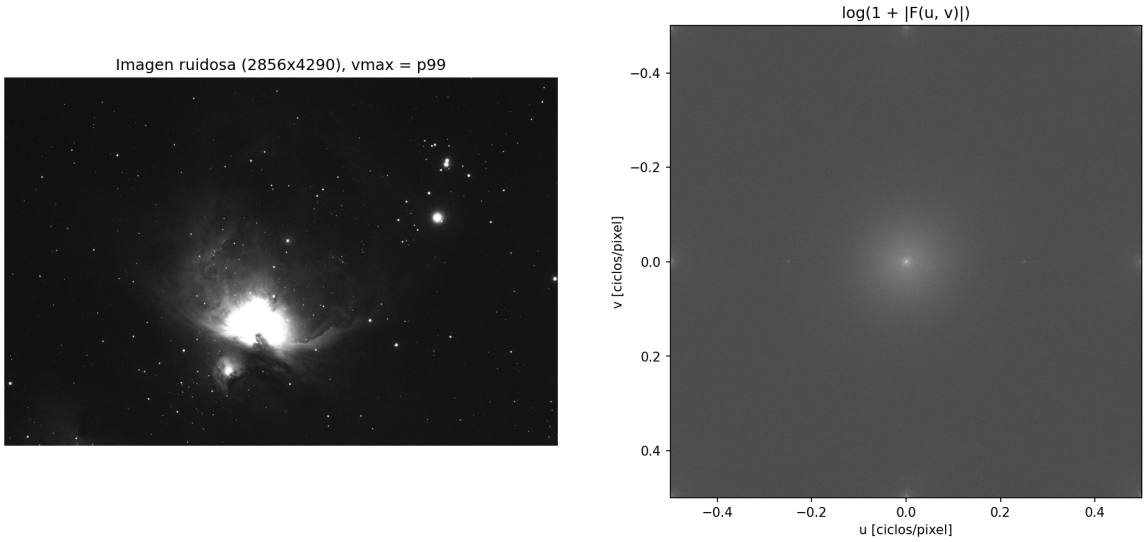

Figura 1.2: Imagen ruidosa (mostrada hasta el percentil 99) y $\log(1 + |F|)$ centrado, sin padding.

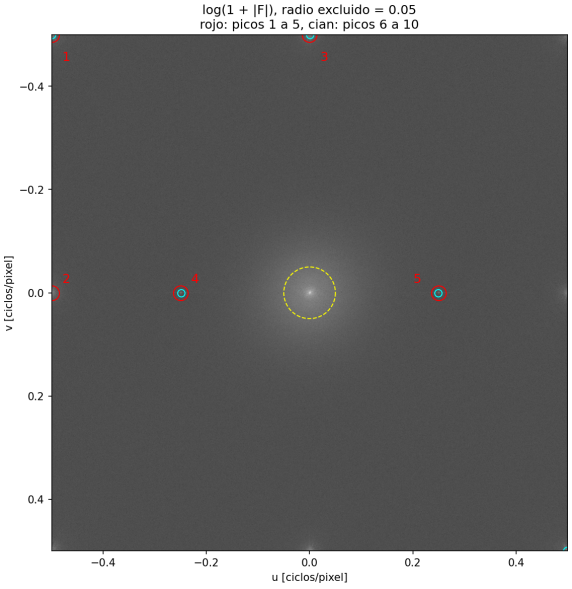

Figura 1.3: Picos encontrados sobre $\log(1 + |F|)$. Rojo: picos 1 a 5. Cian: picos 6 a 10. Amarillo: radio excluido de 0.05.

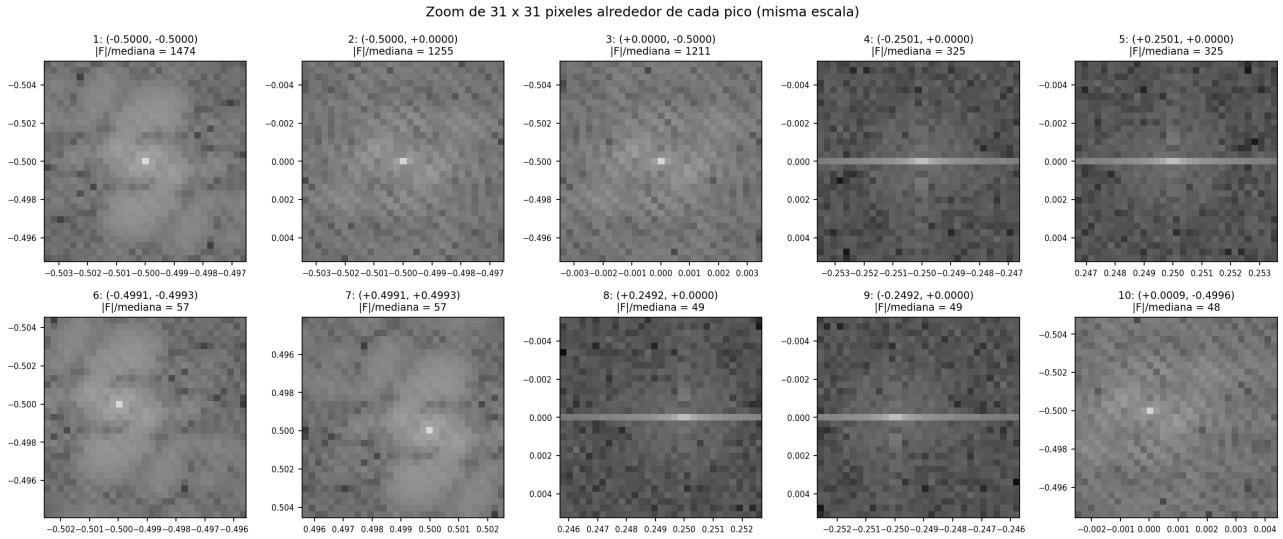

Figura 1.4: Zoom de 31×31 píxeles alrededor de cada pico, con la misma escala. En los picos de ± 0.25 se ve la línea horizontal.

reparte en una línea en u, porque 4290 no es múltiplo de 4 y la frecuencia 0.25 cae entre dos muestras.

Con esto usé los centros $(0.25, 0)$, $(-0.5, -0.5)$, $(-0.5, 0)$ y $(0, -0.5)$, y el simétrico de cada uno se agrega solo. Para el par de columnas puse 0.25 (periodo exacto de 4 píxeles) y no 0.2501, que es solo el píxel más cercano de esta grilla.

1.2. Barrido del ancho del notch

Probé anchos de 0.0005, 0.001, 0.002, 0.005, 0.01, 0.02 y 0.04 (radio en el ideal, σ en el gaussiano), sin padding, porque el padding lo estudié después. El mínimo sale de la resolución en frecuencia, $1/N$ [3]: $1/4290 \approx 0.00023$ en u, y 0.0005 son unos 2 píxeles y un radio menor casi no tapa nada. El máximo tiene que quedar bien bajo 0.125, la mitad de la distancia entre los centros más cercanos, y además lejos del contenido de la imagen ($D < 0.1$). Entre medio usé valores 1-2-5, porque el efecto del ancho cambia por factores y no por sumas.

Para medir el resultado usé tres cosas:

- **Cielo**: en una zona de cielo (filas 100–400, columnas 2400–3200) calculé la mediana de cada columna y de cada fila, y después la desviación estándar de esas medianas. Si queda patrón, las medianas suben y bajan.
- **Zonas brillantes**: en cuatro recortes de 256×256 dentro de la nebulosa sumé $|F|$ en las cinco frecuencias del ruido y lo dividí por la mediana del espectro del recorte. Como 256 es múltiplo de 4, esas frecuencias caen justo en un píxel.
- **std de la diferencia** entre la imagen ruidosa y la filtrada: cuánto se le saca a la imagen.

En el cielo el patrón baja de 0.0088 a $\sim$0.0023 (columnas) y de 0.0084 a $\sim$0.0011 (filas) ya con el ancho más chico, y después casi no cambia nada. En cambio, en las zonas brillantes hace falta un notch más ancho: pasa de 449 a 117 con 0.0005 (ideal), a 33–59 con 0.001–0.002 y llega a $\sim$17 desde 0.005, donde se queda plano. Ese piso sería contenido propio de la imagen en esas frecuencias. Mientras tanto, la std de la diferencia sube siempre (de 0.0181 a 0.0202): desde que el residuo deja de bajar, lo extra que se saca ya es imagen y no ruido. Con el mismo radio de $H = 0.5$, el gaussiano deja el mismo residuo y saca un poco menos que el ideal.

Para ver los casos principales con imágenes elegí 0.001 (angosto), 0.005 (donde las curvas se doblan) y 0.04 (ancho). También agregué una medida de artefactos: el centro de la estrella brillante de arriba a la derecha está saturado y en la imagen original es plano (std $= 0.0036$ en 12×12 píxeles); lo que aparezca ahí después de filtrar lo puso el filtro.

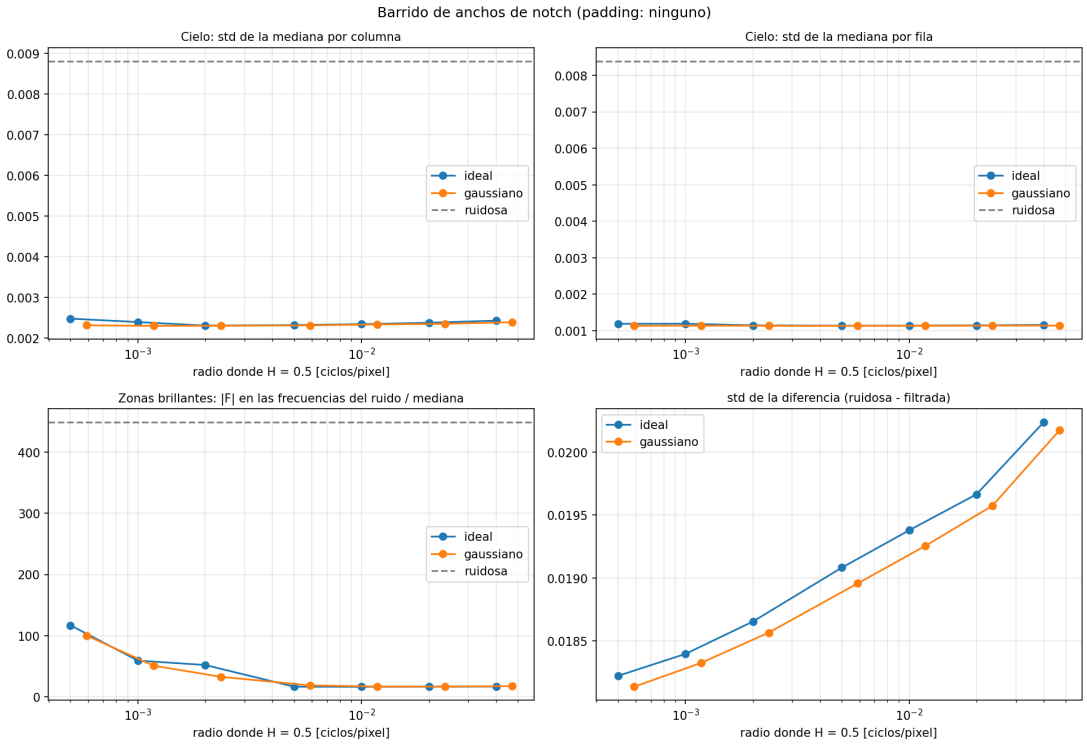

Figura 1.5: Métricas del barrido en función del radio donde $H = 0.5$, sin padding. La línea punteada es la imagen ruidosa.

La Figura 1.6 muestra uno de estos casos (gaussiano con $\sigma = 0.005$) a imagen completa, con el espectro antes y después. En el espectro filtrado aparecen los agujeros en las cinco frecuencias del ruido, y el espectro de la diferencia confirma que solo se sacó lo que está alrededor de ellas. A este tamaño la diferencia se ve casi gris porque la grilla se promedia al achicar la figura, por eso para comparar los casos uso los zooms de las Figuras 1.7 y 1.8.

Ancho	0.001	0.005	0.04
Ideal: std en el núcleo / std diferencia	0.0138 / 0.01840	0.0645 / 0.01908	0.0353 / 0.02024
Gaussiano: std en el núcleo / std diferencia	0.0145 / 0.01833	0.0738 / 0.01896	0.0001 / 0.02018

Tabla 1.2: Artefactos (std dentro del núcleo saturado de la estrella, 0.0036 en la imagen ruidosa) y std de la diferencia en la imagen completa.

Ruido residual. Con 0.001 la grilla del mosaico se sigue viendo en M43 y en el halo de la estrella, y la diferencia es una grilla pareja en todo el zoom: el filtro saca un patrón "promedio" igual en toda la imagen. Con 0.005 la grilla baja pero todavía aparece en la parte más brillante de M43. Con 0.04 casi desaparece, y la diferencia ya no es pareja sino que sigue la forma de la nebulosa. En el halo de la estrella queda algo de grilla en todos los casos.

La razón es que el mosaico de Bayer no es un ruido que se suma: es el color de la escena multiplicado por un patrón 2×2. Donde el color cambia rápido (la estrella, los bordes de la nebulosa), el pico se abre en el espectro, y un notch angosto solo saca la parte pareja del patrón.

Pérdida de detalle. Al ensanchar, la diferencia empieza a mostrar la estructura de M43, o sea que se está sacando imagen además del ruido. Coincide con que la std de la diferencia sube siempre.

Artefactos. Con 0.001 y 0.005 aparece una grilla dentro del núcleo saturado, donde antes no había nada (de 0.0036 a 0.014 y 0.065–0.074): el filtro le resta el patrón promedio también a una zona que no lo tenía. Con 0.04 el gaussiano deja el núcleo plano (0.0001), porque con un agujero ancho lo que se saca está más concentrado en el espacio y sigue lo que pasa localmente. El ideal con 0.04 todavía deja 0.035, por las oscilaciones (ringing) que produce el borde duro del agujero [4]. Con el mismo ancho el ruido residual de los dos filtros es parecido; la diferencia está en los artefactos, y ahí el gaussiano es mejor.

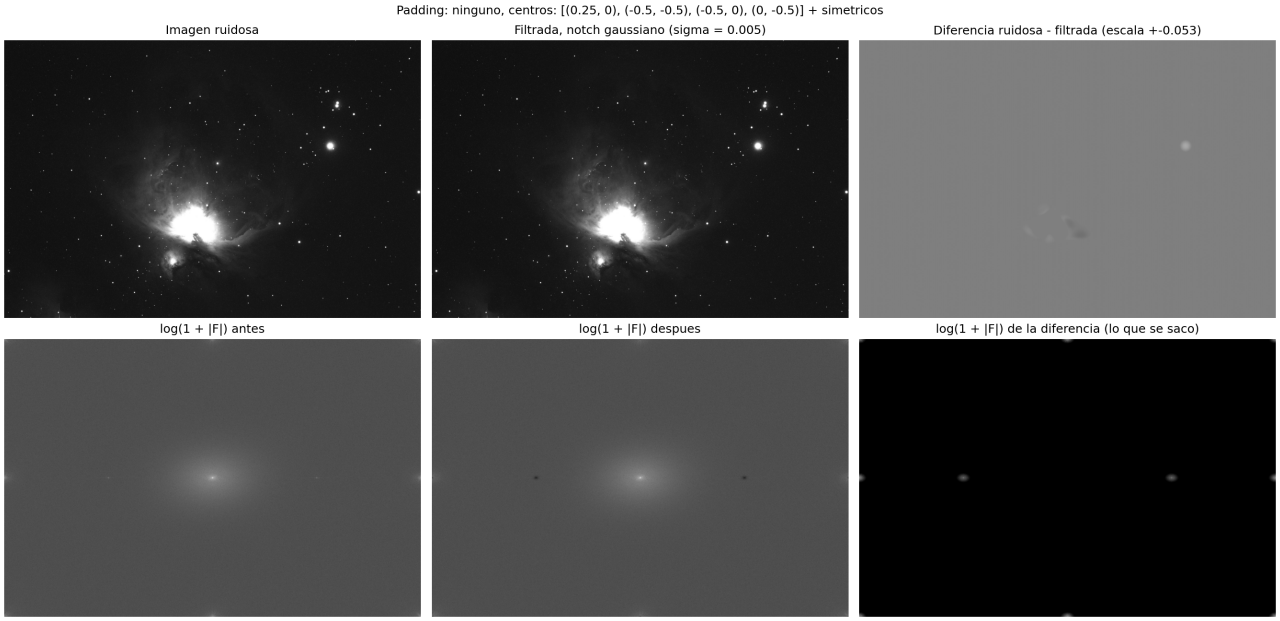

Figura 1.6: Uno de los casos principales a imagen completa: notch gaussiano, $\sigma = 0.005$, sin padding. Arriba: imagen ruidosa, filtrada y diferencia (escala ± 0.053). Abajo: $\log(1 + |F|)$ antes, después y de la diferencia.

Entonces hay un compromiso: un notch angosto saca poca imagen pero mete artefactos donde no había patrón, y uno ancho gaussiano limpia mejor la grilla pero saca más estructura. Así que la decisión final la tomé después de elegir el padding.

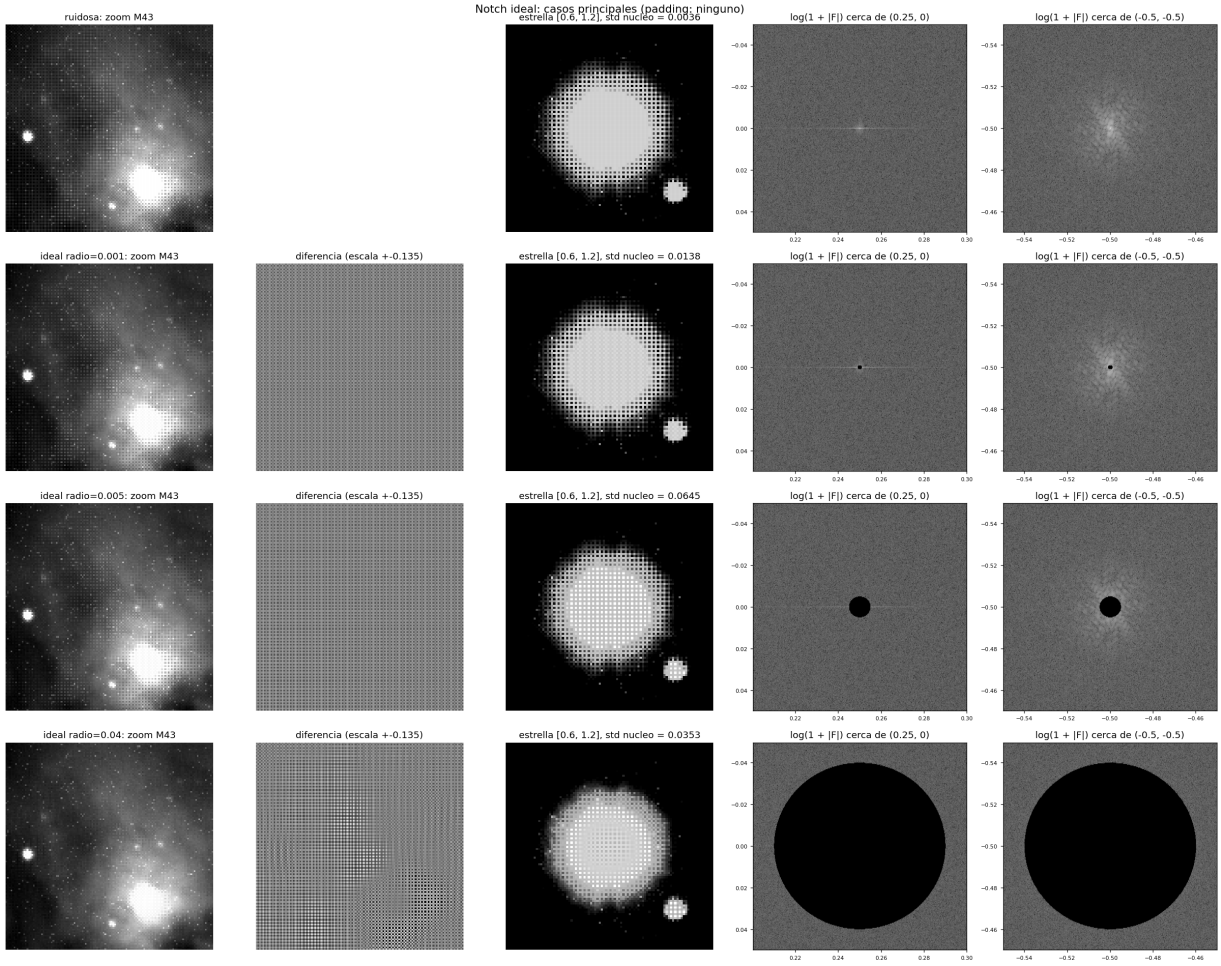

Figura 1.7: Notch ideal sin padding, radios 0.001, 0.005 y 0.04 (filas 2 a 4; la primera fila es la imagen ruidosa). Columnas: zoom de M43 (filas 2180–2308, columnas 1650–1778), diferencia ruidosa $-$ filtrada en ese zoom (escala común ± 0.135), estrella brillante (escala $[0.6, 1.2]$) y espectro filtrado cerca de $(0.25, 0)$ y de $(-0.5, -0.5)$, con la misma escala que el espectro original.

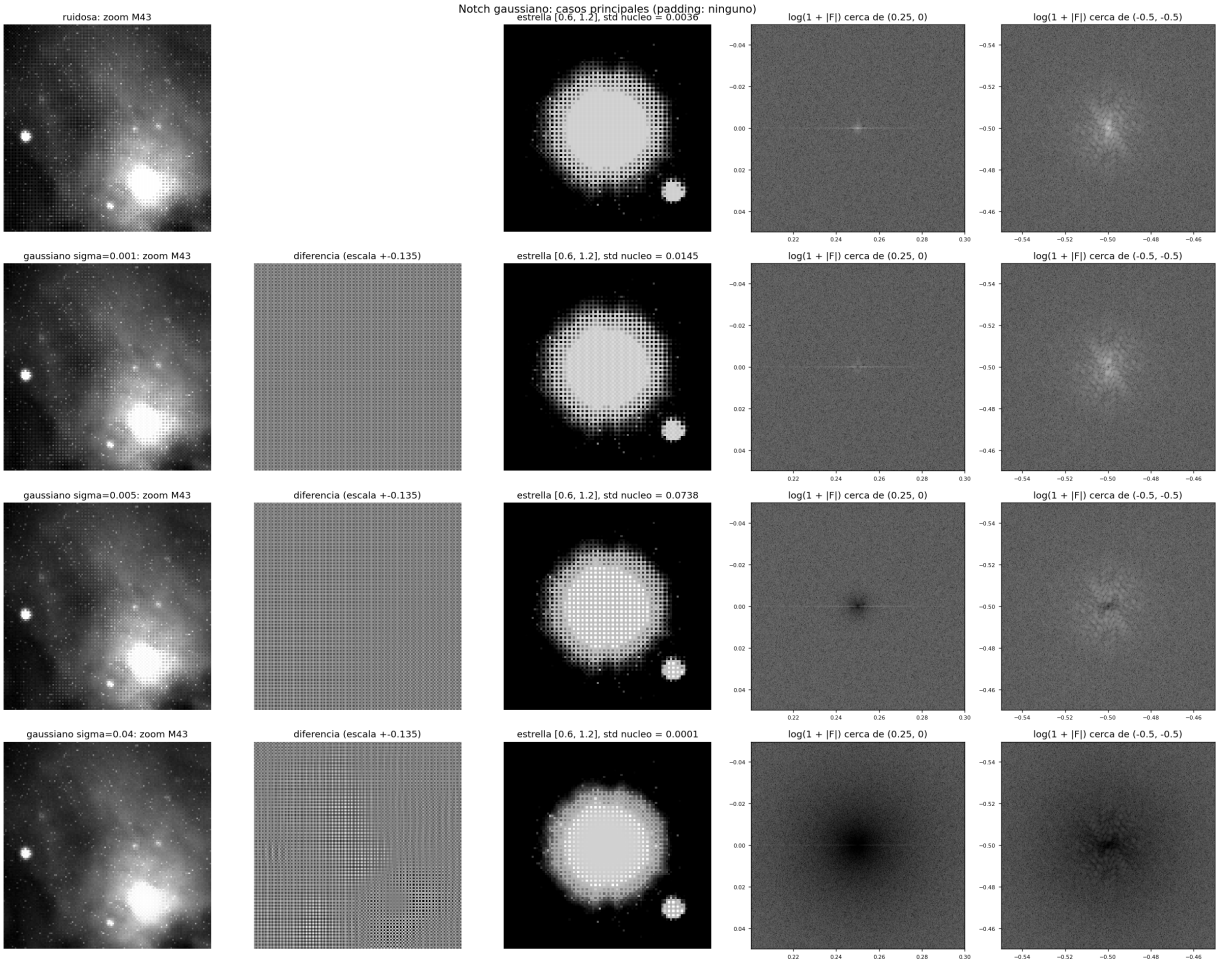

Figura 1.8: Lo mismo que la Figura 1.7 con el notch gaussiano, $\sigma = 0.001$, 0.005 y 0.04.

1.3. Padding

Comparé el filtrado sin padding con padding de ceros, reflejo y borde (`np.pad` con `constant`, `reflect` y `edge`), con 32, 128 y 256 píxeles por lado, usando los dos filtros con ancho 0.005. Elegí esos tamaños pensando en cuánto se extiende el filtro en el espacio: el notch gaussiano saca la frecuencia del ruido multiplicada por una gaussiana de std $s = 1/(2\pi\sigma)$, que con $\sigma = 0.005$ da $s \approx 32$ píxeles. Entonces 32 es $\sim 1s$ (debería quedar corto), 128 es $\sim 4s$ (debería alcanzar) y 256 es $\sim 8s$ (sobra).

Para medir los bordes usé lo mismo que en el cielo, pero en franjas de 48 píxeles pegadas a cada borde: std de la mediana por columna en los bordes izquierdo y derecho, y por fila arriba y abajo. Al medio de la imagen esta medida da $\sim$0.0005 después de filtrar.

Padding	Izquierda	Derecha	Arriba	Abajo
Imagen ruidosa	0.0076	0.0077	0.0088	0.0063
Ninguno	0.0030 / 0.0033	0.0032 / 0.0035	0.0004 / 0.0004	0.0004 / 0.0004
Ceros 128	0.0029 / 0.0031	0.0034 / 0.0037	0.0024 / 0.0029	0.0028 / 0.0030
Borde 128	0.0028 / 0.0030	0.0032 / 0.0036	0.0026 / 0.0030	0.0030 / 0.0035
Reflejo 32	0.0003 / 0.0009	0.0031 / 0.0034	0.0003 / 0.0003	0.0004 / 0.0004
Reflejo 128	0.0003 / 0.0004	0.0030 / 0.0033	0.0003 / 0.0003	0.0004 / 0.0004
Reflejo 256	0.0003 / 0.0003	0.0030 / 0.0033	0.0003 / 0.0003	0.0004 / 0.0004

Tabla 1.3: Patrón que queda en franjas de 48 píxeles en cada borde (gaussiano / ideal, ancho 0.005). Ceros y borde con 32 y 256 dan valores parecidos a 128.

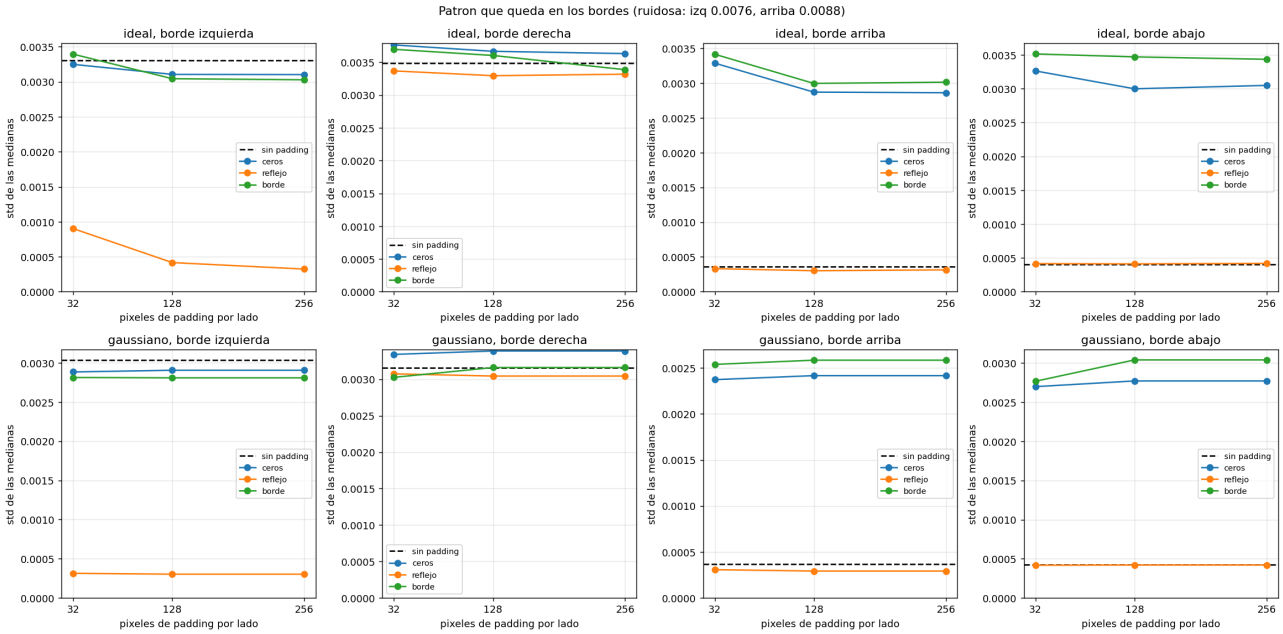

Figura 1.9: La misma medida de la Tabla 1.3 en función del tamaño del padding. Arriba: ideal. Abajo: gaussiano. La línea negra es sin padding.

Sin padding, arriba y abajo quedan bien, pero a la izquierda y a la derecha queda patrón. La FFT trata la imagen como si se repitiera. Y como el alto (2856) es par, al pegar el borde de abajo con el de arriba el patrón de Bayer sigue igual. El ancho (4290) no es múltiplo de 4, entonces al pegar derecha con izquierda el patrón de cada 4 columnas salta de fase, y cerca de esos bordes no se puede sacar bien. Es lo mismo que hacía que el pico de 0.25 se viera ancho en u.

Ceros y borde empeoran arriba y abajo (de 0.0004 a 0.0024–0.0035): lo que se agrega no tiene el patrón, y se crea un corte en los cuatro bordes. Reflejo arregla el borde izquierdo pero no el derecho. El patrón de 4 columnas es simétrico

respecto a las columnas pares; al reflejar en la columna 0 el patrón continúa bien, pero a la derecha el espejo queda en la columna 4289, que es impar, y el patrón queda mal pegado. Con el mismo padding en los cuatro lados no se puede arreglar eso, aunque a la derecha reflejo queda igual que sin padding y no peor.

En cuanto al tamaño, con el gaussiano el reflejo ya funciona con 32. El ideal necesita más (izquierda: 0.0009 con 32, 0.0004 con 128), porque su respuesta en el espacio es de tipo jinc y baja mucho más lento que una gaussiana [4]. Con 256 no se gana nada respecto a 128 y el filtrado tarda un poco más.

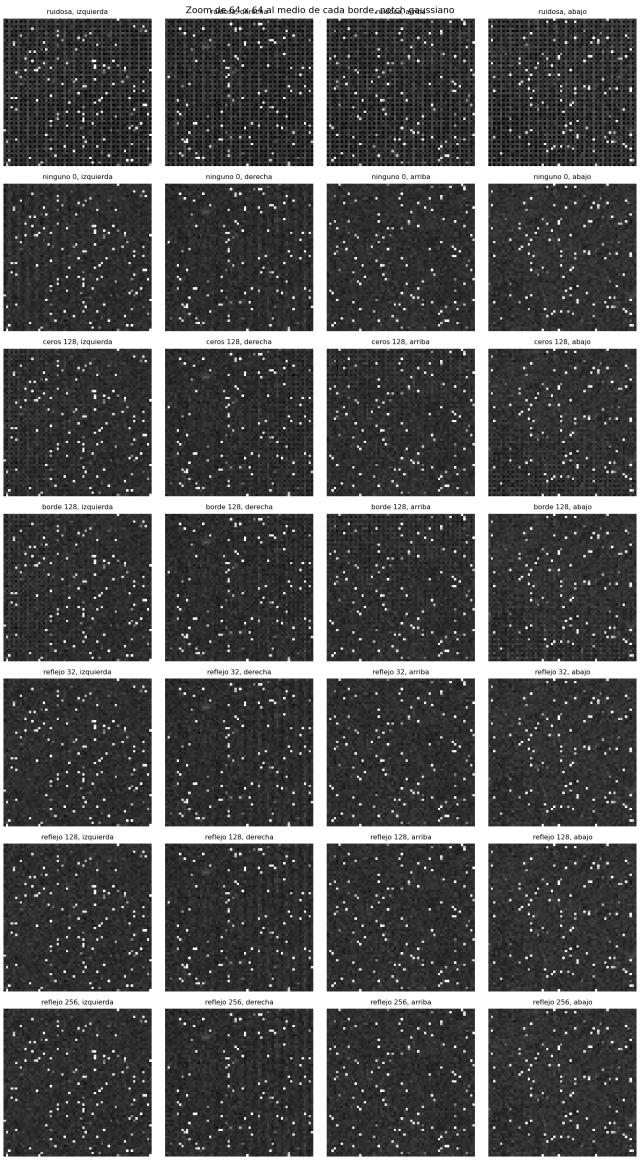

Figura 1.10: Zoom de 64×64 píxeles al medio de cada borde, notch gaussiano con $\sigma = 0.005$. Filas: ruidosa, sin padding, ceros 128, borde 128 y reflejo 32, 128 y 256.

En los zooms de la Figura 1.10 se ve lo mismo: con ceros y borde aparecen rayas arriba y abajo, y con reflejo el borde izquierdo queda limpio. Para el resto de la pregunta elegí **reflejo con 128 píxeles por lado**: es el único que mejora un borde sin empeorar los otros, y 128 alcanza para los dos filtros.

1.4. Configuración final

Antes de elegir repetí el barrido con reflejo 128 y con anchos entre 0.005 y 0.04, porque en la Tabla 1.2 los artefactos cambiaban mucho en ese rango. Esta vez medí los artefactos en cuatro zonas saturadas y no solo en una estrella: el núcleo de la nebulosa, la estrella grande, una estrella más arriba y la estrella de M43 (los píxeles sobre 1 son el 0.73 % de la imagen). En cada una usé una ventana chica que queda 100 % saturada.

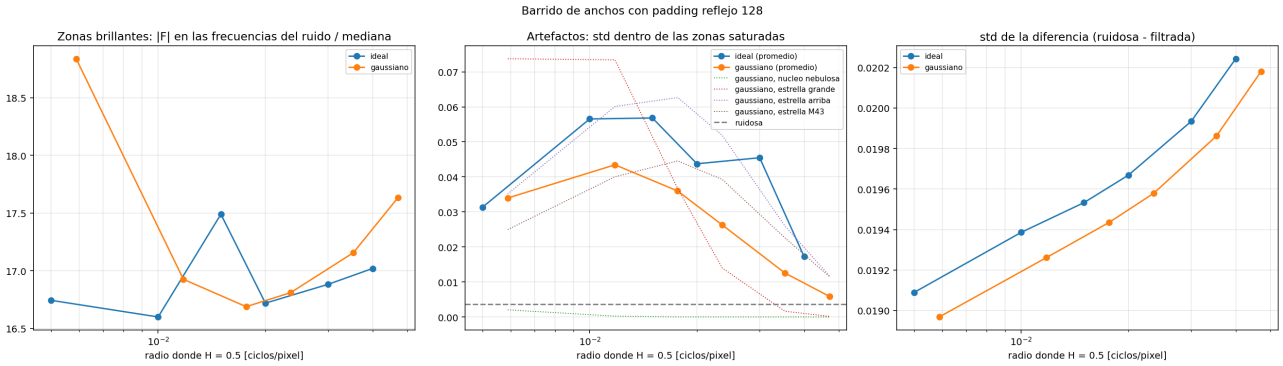

Figura 1.11: Barrido con padding reflejo 128. Izquierda: residuo en las zonas brillantes. Centro: artefactos (promedio de las cuatro zonas saturadas; en línea delgada, cada zona con el gaussiano). Derecha: std de la diferencia.

Ancho	0.005	0.01	0.015	0.02	0.03	0.04
Ideal: artefactos	0.0313	0.0565	0.0568	0.0437	0.0455	0.0172
Gaussiano: artefactos	0.0340	0.0434	0.0360	0.0263	0.0125	0.0059
Ideal: std diferencia	0.01909	0.01939	0.01953	0.01967	0.01993	0.02024
Gaussiano: std diferencia	0.01897	0.01926	0.01944	0.01958	0.01986	0.02018

Tabla 1.4: Barrido con reflejo 128. Artefactos: promedio de la std en las cuatro zonas saturadas (0.0036 en la imagen ruidosa). En las zonas brillantes todos quedan entre 16.6 y 18.8 (449 en la ruidosa).

En las zonas brillantes todos los anchos quedan en el piso y esa métrica ya no sirve para elegir. Lo que queda es el compromiso entre artefactos, que bajan al ensanchar, y lo que se le saca a la imagen, que sube. El ideal sube y baja sin orden (por el ringing) y en su mejor caso deja 0.017, unas tres veces más que el gaussiano con el mismo ancho. Además siempre saca un poco más. Por eso descarté el ideal.

Con el gaussiano dudé entre 0.03 y 0.04. Con 0.04 los artefactos bajan a la mitad, pero se saca más imagen y es el extremo del rango probado. Me quedé con $\boldsymbol{\sigma = 0.03}$: el núcleo de la nebulosa y la estrella grande quedan sin artefactos, se saca menos imagen que con 0.04 y hay datos a los dos lados. La configuración final es **notch gaussiano,** $\sigma = 0.03$ **ciclos/píxel, centros** $(0.25, 0)$**,** $(-0.5, -0.5)$**,** $(-0.5, 0)$ **y** $(0, -0.5)$ **más sus simétricos, padding reflejo de 128 píxeles por lado**.

Métrica	Ruidosa	Filtrada
Cielo, std de la mediana por columna	0.00881	0.00238
Cielo, std de la mediana por fila	0.00839	0.00114
Zonas brillantes, $\|F\|$ en los picos / mediana	449.1	17.2
Artefactos: núcleo de la nebulosa / estrella grande	0.0036 / 0.0036	0.0000 / 0.0016
Artefactos: estrella de arriba / estrella de M43	0.0036 / 0.0036	0.0260 / 0.0226
std de la diferencia	–	0.01986
Promedio de la imagen	0.091652	0.091652

Tabla 1.5: Resultado de la configuración final.

En el espectro filtrado se ven los cinco agujeros, y el espectro de la diferencia muestra que solo se sacan las zonas alrededor de los picos. A imagen completa la diferencia es casi gris, salvo en el núcleo de la nebulosa y en la estrella

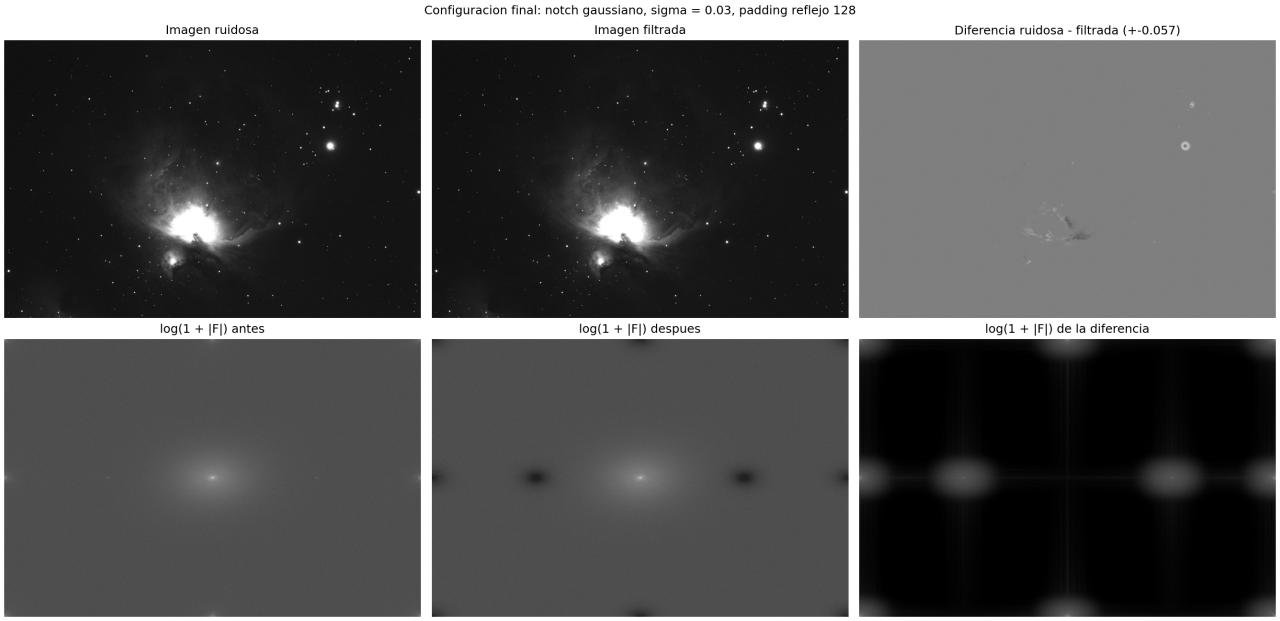

Figura 1.12: Configuración final (gaussiano, $\sigma = 0.03$, reflejo 128). Arriba: imagen ruidosa, filtrada y diferencia (escala ± 0.057). Abajo: $\log(1 + |F|)$ antes, después y de la diferencia.

grande. En los perfiles del cielo desaparece el salto cada 4 columnas y la alternancia cada 2 filas. En el zoom de M43 la grilla casi desaparece y se mantiene el degradado de la nebulosa. El promedio no cambia, porque $H(0, 0) = 1$.

Limitaciones. Quedan artefactos en las estrellas saturadas chicas (std $\sim$0.025) y un poco de grilla en el borde del disco y en el halo de la estrella grande. Y como el mosaico es el color multiplicado por un patrón, donde el color cambia muy rápido un notch no lo puede sacar entero sin sacar también imagen. Igual, las zonas saturadas son solo el 0.73 % de la imagen y ahí no había información real. Además, el borde derecho sigue con algo del patrón de 4 columnas, por el problema de fase que expliqué en el padding.

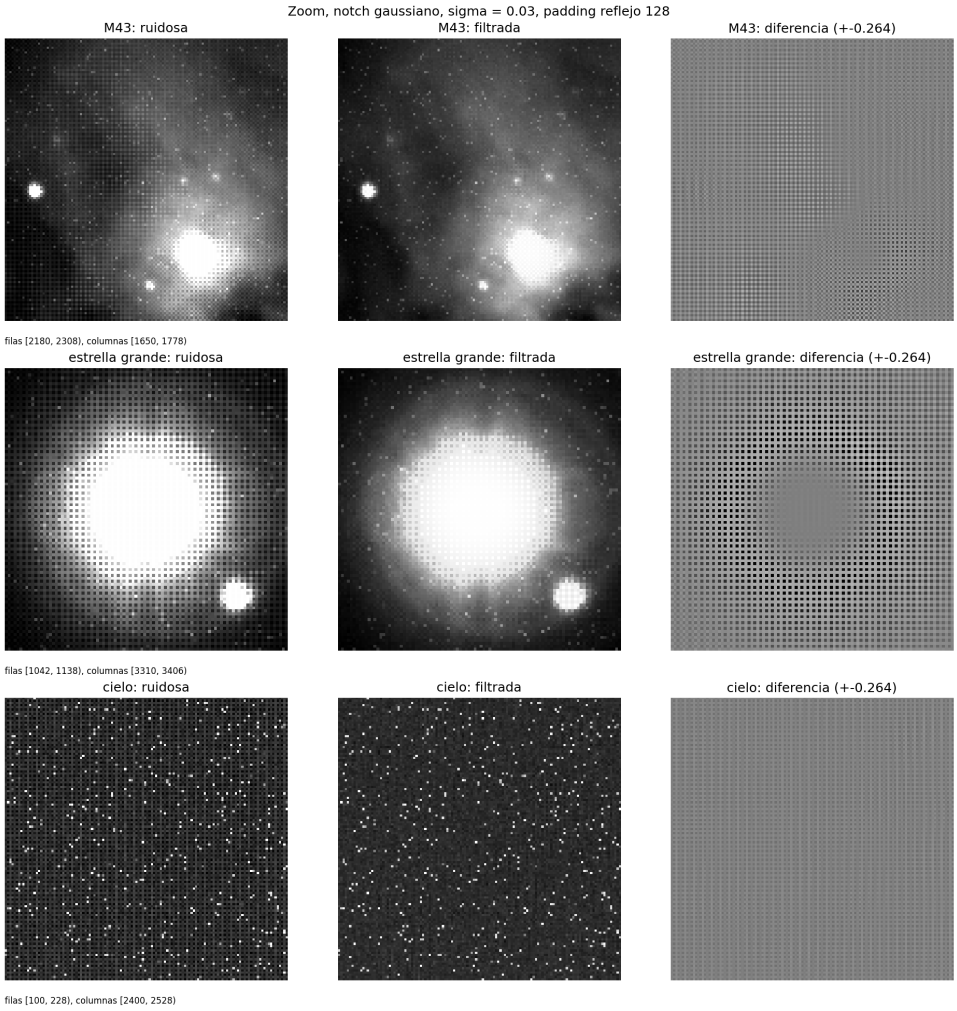

Figura 1.13: Zooms de la configuración final: M43, estrella grande y cielo (coordenadas bajo cada fila). Las diferencias usan la misma escala ± 0.264.

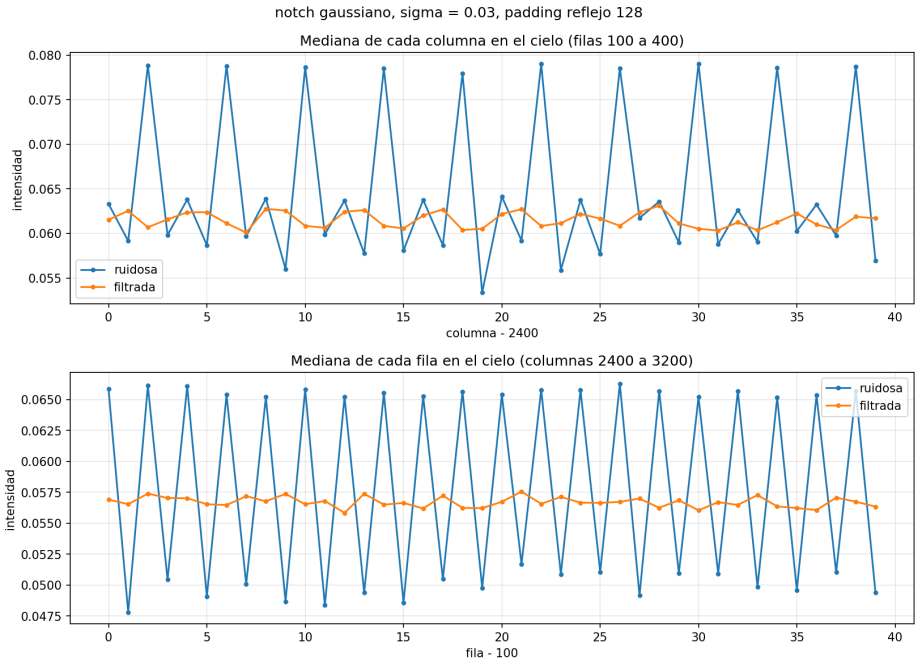

Figura 1.14: Mediana de cada columna (filas 100–400) y de cada fila (columnas 2400–3200) en el cielo, antes y después del filtrado final.

1.5. Respuestas a las preguntas guiadas

Para estas respuestas usé la configuración final. En `test_p1_preguntas` hice la cadena a mano, paso a paso, para tener F y H disponibles, y revisé que diera exactamente lo mismo que `filtrar_notch` (diferencia máxima 0).

1 Simetría del espectro

Una perturbación sinusoidal real se puede escribir como $\cos\theta = \frac{1}{2}\left(e^{j\theta} + e^{-j\theta}\right)$: son dos exponenciales con frecuencias opuestas, por eso aparecen dos picos en $\pm(u_0, v_0)$. En general, para cualquier imagen real se cumple $F(-u,-v) = F^*(u,v)$. En los datos, en el pico de 0.25 se obtiene $|F(-u,-v) - F^*(u,v)| = 1.4 \times 10^{-11}$, con $|F| \approx 34669$, o sea error numérico. En el pico de Bayer $(-0.5,-0.5)$ el simétrico es el mismo píxel y por eso F sale real $(-156862.94{+}0j)$.

En el código incorporé la simetría en `agregar_simetricos`: por cada centro que se le da agrega $(-u_0,-v_0)$, salvo cuando cada coordenada es 0 o ± 0.5. Así los notches siempre van en pares y H queda simétrico.

2 Coordenadas de frecuencia y distancia al notch

`crear_grilla_frecuencias` arma U y V en ciclos/píxel con `fftfreq` y `fftshift`, en el mismo orden que el espectro centrado. Así, cada píxel del espectro tiene su (u, v) en la misma posición de U y V. Después `distancia_a_centro` calcula la distancia de la Ecuación (1.1): diferencia en cada eje, corregida con $\min(d, 1-d)$ porque el espectro es periódico, y la norma de las dos.

La distancia es la misma para los dos filtros; lo que cambia es cómo se usa. En `notch_ideal` se compara $D \le r$ y se pone $H = 0$ dentro del círculo. En `notch_gaussiano` se calcula $1 - e^{-D^2/2\sigma^2}$, que cae suave hacia 0 en el centro. Con varios centros, en los dos casos se multiplican los filtros de cada uno.

3 Padding y recorte

`aplicar_padding` usa `np.pad` con el mismo tamaño en los cuatro bordes. Con reflejo (`mode="reflect"`) pone afuera los píxeles de adentro en espejo sin repetir el borde: la columna -1 es la columna 1, la -2 es la 2, y así (lo revisé en el script). Con 128 píxeles por lado la imagen pasa de 2856×4290 a 3112×4546, y H se construye con ese tamaño. Después de la `ifft`, `recortar_padding` toma las filas $[128, 2984)$ y las columnas $[128, 4418)$. Revisé que ese recorte aplicado a la imagen con padding (antes de filtrar) devuelve exactamente la original.

4 Por qué se filtra el espectro complejo

Por el teorema de convolución, filtrar es multiplicar el espectro complejo por H, y eso es lo que hice. Para verlo comparé tres versiones en el zoom de M43: (a) $F \cdot H$; (b) H aplicado solo a la magnitud, dejando la fase igual, $|F|He^{j\angle F}$; y (c) H aplicado a la magnitud y también a la fase, $|F|He^{jH\angle F}$.

(a) y (b) dan lo mismo (diferencia máxima 8.9×10^{-16}). Como H es real y no negativo, multiplicar F por H solo cambia la magnitud y separar no aporta nada. (c) sí es distinto (diferencia máxima 0.035): donde $0 < H < 1$ la fase cambia y esas componentes se corren de lugar, porque la fase indica dónde está cada onda [1]. A simple vista (c) se parece, pero la resta de la Figura 1.15 muestra grilla corrida alrededor de la nebulosa. Filtrar magnitud y fase por separado solo tendría sentido si se hiciera exactamente como en (b), y eso ya es lo mismo que multiplicar el complejo.

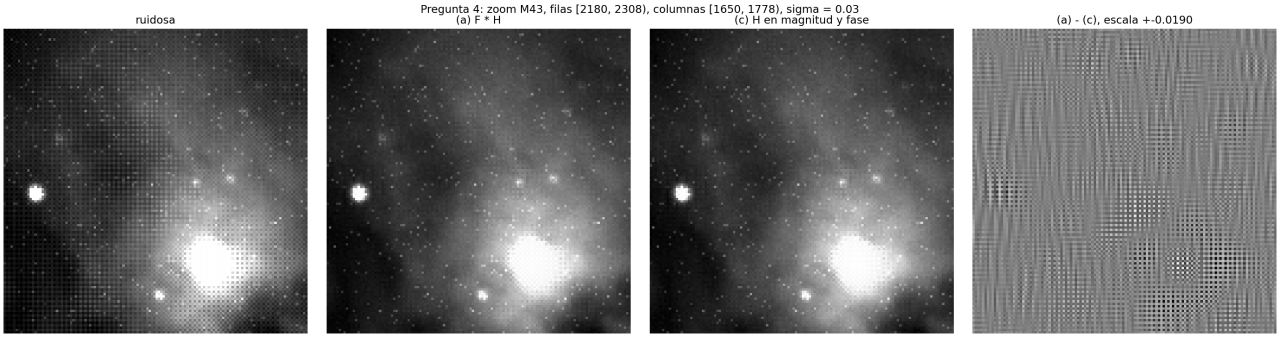

Figura 1.15: Zoom de M43 (filas 2180–2308, columnas 1650–1778) con la configuración final: ruidosa, (a) $F \cdot H$, (c) H en magnitud y fase, y (a) $-$ (c) con escala ± 0.019.

5 Seguimiento de una frecuencia

Tomé tres frecuencias de la imagen con padding (3112×4546): el pico de 0.25, un punto en el borde del agujero, a una distancia σ del centro, y el pico de Bayer de la esquina.

Frecuencia (u, v)	F antes	H	$F \cdot H$
$(+0.24989, 0)$, pico	$-2699.49 - 34564.10j$	0.000007	$-0.0181 - 0.2323j$
$(+0.28003, 0)$, borde	$-66.09 + 133.60j$	0.394003	$-26.04 + 52.64j$
$(-0.5, -0.5)$, Bayer	$-156862.94 + 0j$	0.000000	≈ 0

Tabla 1.6: Valor del espectro antes y después de multiplicar por H (gaussiano, $\sigma = 0.03$, reflejo 128).

En los tres casos la fase no cambia (-1.6487 y $+2.0302$ rad en los dos primeros), solo la magnitud, que queda multiplicada por H. En el pico la magnitud baja de 34669 a 0.233. En el borde $H = 0.394 \approx 1 - e^{-1/2}$, justo lo que da el gaussiano a una distancia σ. El pico de 0.25 queda en $u = 0.24989$ y no en 0.25 porque 4546 tampoco es múltiplo de 4. Su simétrico en -0.24989 tiene el valor conjugado y recibe el mismo H.

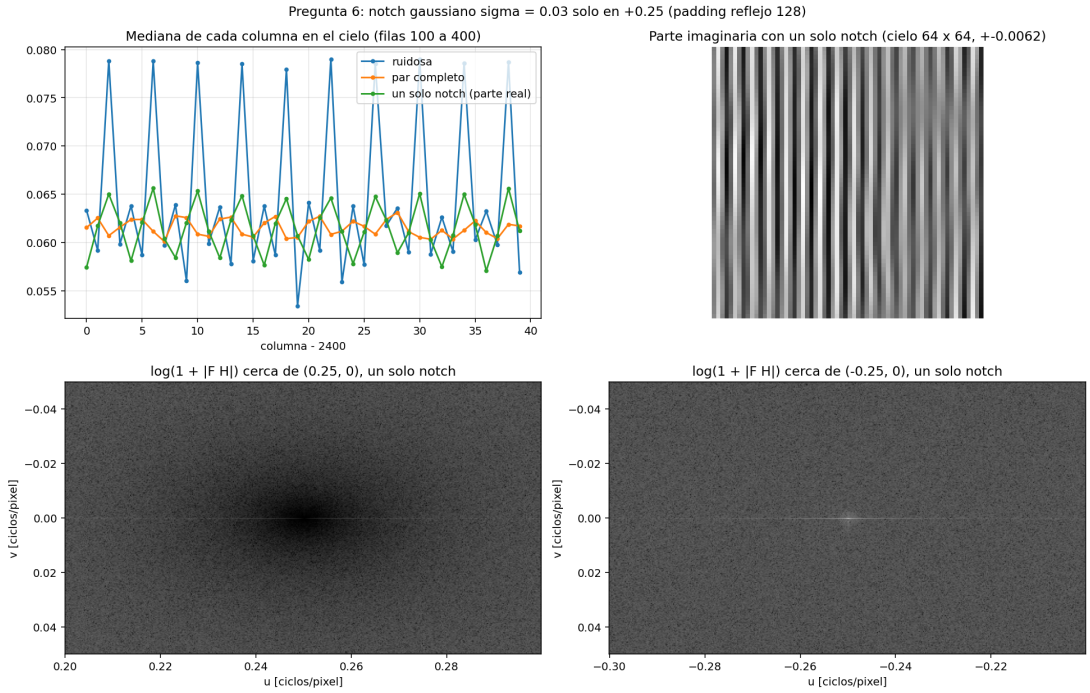

Figura 1.16: Un solo notch en $+0.25$ (gaussiano, $\sigma = 0.03$, reflejo 128). Arriba: mediana por columna en el cielo y parte imaginaria en un recorte de cielo de 64×64. Abajo: $\log(1 + |FH|)$ cerca de $+0.25$ (con agujero) y de -0.25 (sin agujero).

6 Qué pasa si se elimina solo un notch del par

Armé H con los agujeros de Bayer y un solo agujero en $+0.25$, sin el de -0.25. La parte imaginaria máxima después de la `ifft` pasa de 1.3×10^{-15} (par completo) a 1.8×10^{-2}. Como H ya no es simétrico, $F \cdot H$ deja de cumplir $F(-u, -v) = F^*(u, v)$ y la imagen resultante ya no es real. En la Figura 1.16, esa parte imaginaria tiene rayas verticales cada 4 columnas: es la exponencial que quedó en -0.25.

Si me quedo con la parte real, el patrón sigue ahí, pero más chico: en el cielo la std de la mediana por columna queda en 0.00356, contra 0.00881 en la ruidosa y 0.00238 con el par completo. Esto calza con $\cos\theta = \frac{1}{2}(e^{j\theta} + e^{-j\theta})$: se sacó una de las dos exponenciales y la parte real de la que queda es un coseno con la mitad de la amplitud.

Código y material de referencia

Implementación: `src/pregunta1.py`. Experimentos, en el orden en que se hicieron: `test_p1_espectro`, `test_p1_perfiles`, `test_p1_picos`, `test_p1_filtros_notch`, `test_p1_filtrado`, `test_p1_barrido`, `test_p1_casos`, `test_p1_padding`, `test_p1_barrido_padding`, `test_p1_final` y `test_p1_preguntas`, en `experiments`. El README indica cómo ejecutarlos y `informe/notas.md` tiene las notas de cada paso.

2. Deconvolución de Wiener y Tikhonov iterativo (p2)

Para esa pregunta implementé el filtro de Wiener simplificado y Tikhonov por descenso de gradiente, con el mismo modelo de convolución circular en todo. Degradé `cameraman.jpg` con la PSF entregada y ruido gaussiano de $\sigma = 0.01$ y $\sigma = 0.05$. El filtro inverso ($K = 0$) no sirve con ruido, pero con el K óptimo Wiener baja el RMSE de 0.0807 a 0.0659 y de 0.0944 a 0.0786. Tikhonov iterativo con $\lambda = K$ llega a la misma solución que Wiener (diferencia del orden de 10^{-7} o menos), y con $\lambda = 0$ solo sirve si se detiene en la iteración correcta.

Datos y modelo de degradación

`cameraman.jpg` es de 256×256 con tres canales iguales; la leí con `cargar_imagen` de la P1, que la pasa a gris en $[0, 1]$. `psf.png` es de 15×15 y sumaba 35.45, por eso `cargar_psf` la divide por su suma. Con eso $H(0,0) = 1$ y la degradación no cambia el promedio de la imagen (0.465586 antes y después). El máximo de la PSF está en $(7, 7)$ y su centro de masa en $(7.002, 7.006)$, entonces el centro es $(15//2, 15//2) = (7, 7)$. La PSF no es exactamente simétrica: $\max |h - h_{\text{girada } 180^\circ}| = 2.2 \times 10^{-4}$, un 0.78 % del máximo.

Usé convolución circular en toda la pregunta, igual que en la cápsula de la semana 10 [1]. Además, la solución cerrada de Tikhonov se escribe como un Wiener en Fourier porque la DFT diagonaliza la convolución [8], y eso es exacto solo con convolución circular. Así puedo comparar la solución iterativa con la cerrada sin que el borde meta diferencias. La limitación es que una foto real no es periódica [7]: en $h * x$ el cielo de la primera fila se oscurece hasta 0.23 por las piernas de la última fila. Como y se genera con el mismo modelo, esto no aparece como error en la restauración, así que es un caso más favorable que una foto real.

H se construye en `psf_a_otf`, como `psf2otf` de la cápsula: (1) pongo la PSF en la esquina de una matriz de ceros de 256×256 (ajuste de tamaño); (2) la muevo con `np.roll` en $(-7, -7)$ para que su centro quede en el píxel $(0, 0)$ (centrado); (3) calculo `fft2` sin `fftshift`, para que H quede en el mismo orden que `fft2(x)`. `aplicar_H` multiplica en frecuencia y vuelve con `ifft2`. Revisé que diera lo mismo que una convolución circular hecha a mano (suma de las 225 copias de x desplazadas con `np.roll`): la diferencia máxima es $\sim 10^{-15}$.

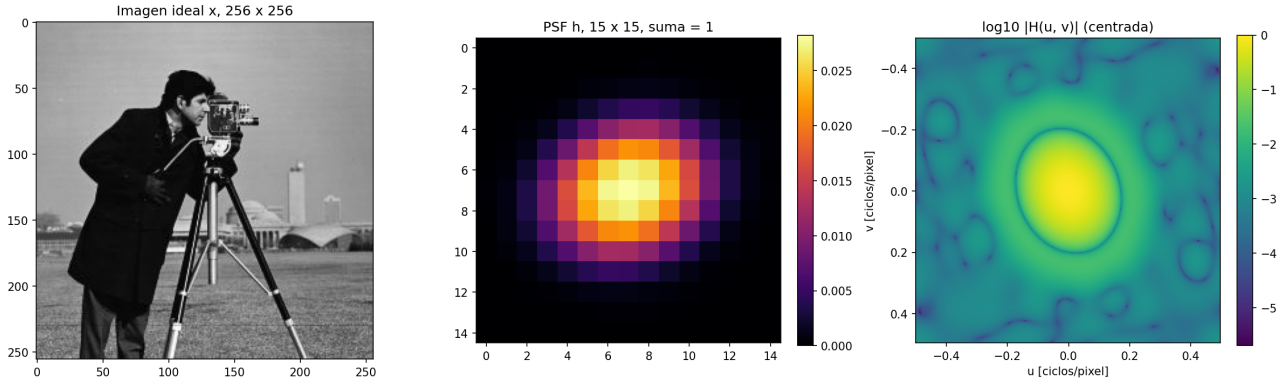

Figura 2.1: Imagen ideal x, PSF normalizada y $\log_{10} |H(u, v)|$ (centrada solo para mostrarla).

H es un pasa bajos: $|H|$ vale 1 en $(0, 0)$, cae casi a cero cerca de 0.17 ciclos/píxel en u y 0.2 en v (la PSF es un poco elíptica), y llega a un mínimo de 2×10^{-6}. $|H| < 0.1$ en el 93 % de las frecuencias, < 0.01 en el 78 % y < 0.001 en el 48 %.

La observación la genera `degradar`: $y = h * x + \sigma\, n$, con n gaussiano de media 0 generado con `np.random.default_rng(semilla)`, como en la cápsula. Usé semilla 0 en toda la pregunta, así el ruido es el mismo en todos los experimentos. No recorté y a $[0, 1]$, porque así el ruido sigue siendo gaussiano y sumado, como en el modelo $g = Hf + n$ [8]. Para medir usé $\text{RMSE}(\hat{x}, x) = \sqrt{\text{MSE}}$, con el MSE de la clase 16 [7], sobre la imagen completa y sin recortar $\hat{x}$.

Implementación de Wiener (parte A)

`wiener(y, H, K)` calcula Y con `fft2`, aplica

$$\hat{X}(u, v) = \frac{H^*(u, v)\, Y(u, v)}{|H(u, v)|^2 + K}, \tag{2.1}$$

y vuelve con `ifft2`, quedándose con la parte real. Con $K = 0$ queda $H^*/|H|^2 = 1/H$, o sea el filtro inverso, y no necesité otra función. Como el mínimo de $|H|^2$ es 4×10^{-12} y no 0, con $K = 0$ no se divide por cero.

Implementación de Tikhonov (parte B)

`tikhonov_gd` minimiza $J(x) = \frac{1}{2}\|h * x - y\|_2^2 + \frac{\lambda}{2}\|x\|_2^2$ con

$$x^{k+1} = x^k - \tau \left[H^T(Hx^k - y) + \lambda x^k \right]. \tag{2.2}$$

H y H^T se aplican en frecuencia con el mismo modelo circular: `aplicar_H` multiplica por H y `aplicar_Ht` por H^* [8], sin armar ninguna matriz. La función recibe λ, τ, el número máximo de iteraciones y la tolerancia del criterio de detención, parte desde $x^0 = y$ (como Landweber en la clase 17) y guarda por iteración el costo, el cambio relativo y, si se le pasa la imagen ideal, el RMSE. Se detiene cuando

$$\frac{\|x^{k+1} - x^k\|_2}{\|x^k\|_2} < \text{tol}, \tag{2.3}$$

que es el criterio de la clase 17, cuando llega al máximo de iteraciones o si el costo deja de ser finito.

Para elegir τ miré la iteración en Fourier. Cada frecuencia cumple $X^{k+1} = \big(1 - \tau(|H|^2 + \lambda)\big) X^k + \tau H^* Y$. Igual que la condición de convergencia de Van Cittert de la clase 17, para que el error no crezca hay que pedir $|1 - \tau(|H|^2 + \lambda)| < 1$ en todas las frecuencias. Como $\max |H| = H(0,0) = 1$, queda

$$0 < \tau < \frac{2}{1 + \lambda}. \tag{2.4}$$

La frecuencia más lenta es la que tiene $|H| \approx 0$: se multiplica por $1 - \tau\lambda$ en cada iteración.

2.1. Niveles de ruido

Para elegir los dos niveles probé 0.002, 0.01 y 0.05, que son los de la cápsula (poco, moderado y mucho ruido), y 0.005 y 0.02 entre medio. Además del RMSE calculé el SNR de la clase 16, $\sum |F|^2 / \sum |N|^2$, y un "cruce": comparé el promedio de $|HX|^2$ en anillos de frecuencia con el nivel del ruido blanco, que en promedio vale $\sigma^2 MN$ en cada frecuencia, y busqué el primer anillo donde la señal borrosa queda bajo el ruido (Figura 2.3).

σ	RMSE(y, x)	SNR	SNR [dB]	Cruce [ciclos/píxel]	y fuera de $[0,1]$
0.002	0.0801	69191	48.4	0.205	0 %
0.005	0.0802	11071	40.4	0.185	0 %
0.01	0.0807	2768	34.4	0.165	0 %
0.02	0.0825	692	28.4	0.155	0.10 %
0.05	0.0944	111	20.4	0.125	2.13 %

Tabla 2.1: Niveles de ruido probados (semilla 0). Sin ruido, RMSE$(h * x, x) = 0.0801$.

Sin ruido el RMSE ya es 0.0801, así que hasta $\sigma = 0.02$ casi todo el error de y es por el desenfoque. El cruce se mueve poco (de 0.205 a 0.125) aunque σ cambie 25 veces, porque $|HX|^2$ cae muy rápido entre 0.1 y 0.2, donde H llega a su primer cero. Pasado $\sim$0.2 la señal borrosa queda bajo el ruido con cualquier σ de los probados, así que esas frecuencias no se van a poder recuperar.

Elegí $\boldsymbol{\sigma = 0.01}$ y $\boldsymbol{\sigma = 0.05}$. Son dos niveles de la cápsula y son claramente distintos: 34.4 dB contra 20.4 dB, cruce en 0.165 contra 0.125. Con 0.01 el ruido apenas se nota en el zoom y el error es casi todo desenfoque; con 0.05 el ruido se ve fuerte y ya aporta una parte importante del error. 0.002 casi no tiene ruido (48 dB).

Figura 2.2: Imagen ideal y degradadas con los cinco niveles de ruido (semilla 0). Abajo: zoom de la cara y la cámara (filas 30–110, columnas 100–200). Escala $[0, 1]$ en todas.

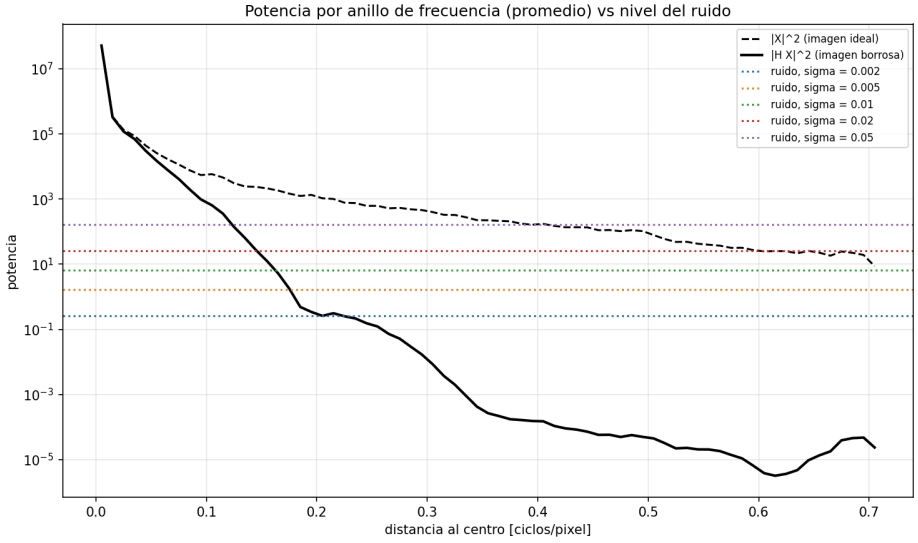

Figura 2.3: Promedio por anillo de $|X|^2$ y $|HX|^2$, y nivel medio del ruido para cada σ.

2.2. Filtro de Wiener y barrido de K

Primero probé el filtro inverso ($K = 0$). Sin ruido recupera x casi exacta (RMSE 3.6×10^{-13}), pero con $\sigma = 0.01$ el RMSE es 42.3 y la imagen va de -155 a 172, y con $\sigma = 0.05$ el RMSE es 211 (Figura 2.4). El error sube 5 veces cuando σ sube 5 veces, porque es solo el ruido dividido por H. Esto lo explico con más detalle en la pregunta guiada 1.

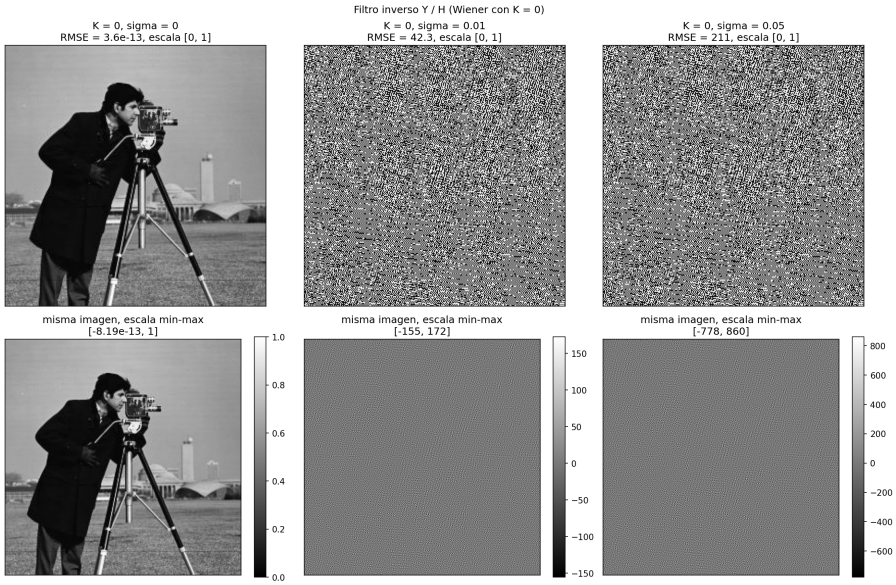

Figura 2.4: Filtro inverso ($K = 0$) sin ruido, con $\sigma = 0.01$ y con $\sigma = 0.05$. Arriba: escala $[0, 1]$. Abajo: la misma imagen con escala mínimo–máximo.

Después hice un barrido de K con `np.logspace(-7, 1, 61)`, el mismo rango del panel de Wiener de la cápsula (unos 7.5 valores por década). Alcanza, porque $|H|^2$ va de 4×10^{-12} a 1: con K menor a 10^{-4} el resultado ya se parece al inverso y con K mayor a 1 se apaga casi todo. Para ver poca y mucha regularización usé $K_{\text{opt}}/100$ y $K_{\text{opt}} \times 100$.

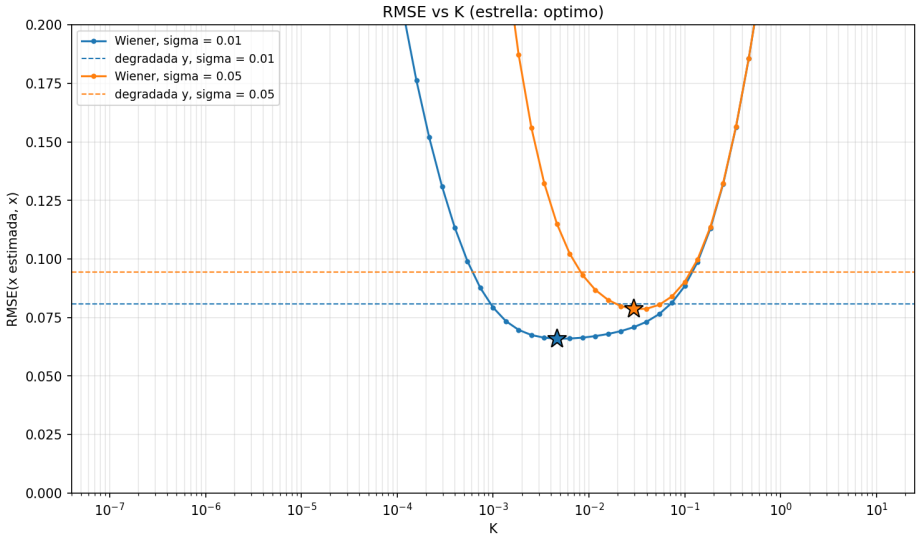

Figura 2.5: RMSE vs. K para $\sigma = 0.01$ y $\sigma = 0.05$, semilla 0 (61 valores entre 10^{-7} y 10). La estrella marca el óptimo y la línea punteada el RMSE de la degradada.

σ	RMSE y	K_{opt}	RMSE con K_{opt}	con $K_{\text{opt}}/100$	con $K_{\text{opt}} \times 100$	$1/\text{SNR}$
0.01	0.0807	4.64×10^{-3}	**0.0659**	0.3234	0.1855	3.6×10^{-4}
0.05	0.0944	2.93×10^{-2}	**0.0786**	0.6044	0.3974	9.0×10^{-3}

Tabla 2.2: Resultado del barrido de K.

Las dos curvas tienen forma de U: a la izquierda domina el ruido amplificado y a la derecha la sobre-regularización. Solo en una franja de K (más o menos de 10^{-3} a 5×10^{-2} con $\sigma = 0.01$, y de 9×10^{-3} a 10^{-1} con $\sigma = 0.05$) la imagen restaurada es mejor que y. Con más ruido el óptimo se corre a la derecha, lo que calza con la clase 16: K hace el papel de S_η/S_f y más ruido pide más regularización [7]. De hecho K_{opt} queda en el orden de $1/\text{SNR}$ o un poco más arriba (13 y 3 veces). Pero K_{opt} crece 6.3 veces y no $25 = (0.05/0.01)^2$: un K constante no puede seguir a S_η/S_f, que cambia con la frecuencia, así que el óptimo es un compromiso entre todas.

Figura 2.6: Wiener con $\sigma = 0.01$: ideal, degradada y $K = K_{\text{opt}}/100$, K_{opt} y $K_{\text{opt}} \times 100$. Abajo: zoom de la cara y la cámara. Escala $[0, 1]$.

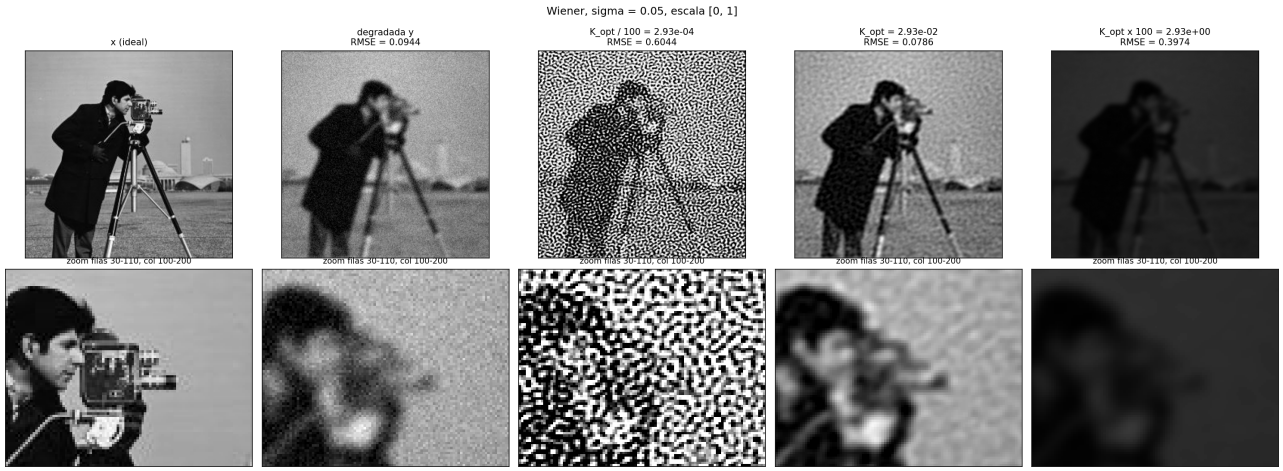

Figura 2.7: Lo mismo que la Figura 2.6 con $\sigma = 0.05$.

Con $K_{\text{opt}}/100$ la imagen se llena de manchas de ruido amplificado. Con K_{opt} queda más nítida que y (se recuperan los bordes del abrigo, el trípode y la cámara), pero en el cielo queda una textura de manchas, que es ruido de las frecuencias donde W todavía amplifica; con $\sigma = 0.05$ esa textura es más gruesa y la imagen queda más suave. Con $K_{\text{opt}} \times 100$ queda borrosa y además más oscura: $W(0,0) = 1/(1+K)$, que con $K = 2.93$ deja el brillo en $\sim 1/4$. El término K (o $\lambda\|x\|^2$) también castiga el promedio de la imagen, no solo el ruido.

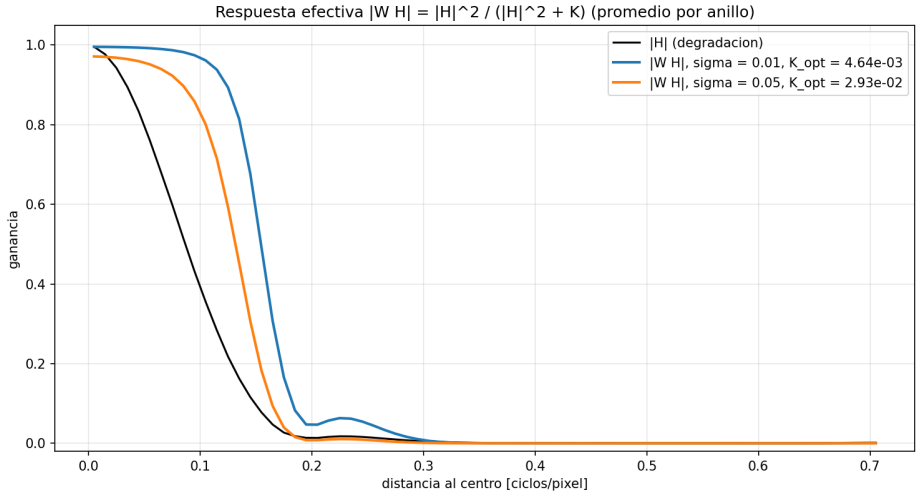

Figura 2.8: Respuesta efectiva $|WH| = |H|^2/(|H|^2 + K_{\text{opt}})$, promedio por anillo, para los dos niveles de ruido.

La respuesta efectiva $|WH|$ (Figura 2.8) es ≈ 1 en las frecuencias bajas, que se recuperan, y ≈ 0 en las altas, que se apagan. Cae a 0.5 cerca de 0.155 ($\sigma = 0.01$) y 0.13 ($\sigma = 0.05$), casi donde estaba el cruce entre señal y ruido (0.165 y 0.125). O sea, el K óptimo apaga más o menos justo donde el ruido pasa a la señal. Como pasado $\sim$0.2 no hay señal que recuperar, la mejora sobre y es moderada: un 18 % y un 17 %.

2.3. Wiener con una PSF ligeramente incorrecta

Para este experimento usé $\sigma = 0.01$, porque con poco ruido el efecto del error en la PSF no queda tapado por el ruido. La observación y siempre se hace con la PSF correcta y solo cambio la PSF que uso para restaurar. Elegí perturbaciones del tipo que se ven en la deconvolución semi-ciega de la clase 15, donde se conoce la forma de la PSF pero no exactamente su centro o su tamaño [6]:

- **Corrida 1 píxel**: `np.roll(psf, 1, axis=1)`, como un error de centrado.
- **Ancha** $\times 1.1$ **y angosta** $\times 0.9$: la PSF escalada alrededor de su centro con `cv2.getRotationMatrix2D` y `cv2.warpAffine` (interpolación bilineal [5]), del mismo tamaño 15×15 y normalizada otra vez a suma 1.

Restauré con el K óptimo de la PSF correcta (4.64×10^{-3}) y además repetí el barrido de K con cada PSF.

PSF usada	$\|h_{\text{mal}} - h\|/\|h\|$	RMSE con $K = 4.64 \times 10^{-3}$	K_{opt} propio	RMSE con su K_{opt}
Correcta	0	0.0659	4.64×10^{-3}	0.0659
Corrida 1 píxel	0.291	0.0771	8.58×10^{-3}	0.0765
Ancha $\times 1.1$	0.164	0.0684	6.31×10^{-3}	0.0682
Angosta $\times 0.9$	0.139	0.0679	6.31×10^{-3}	0.0677

Tabla 2.3: Wiener con PSF incorrectas ($\sigma = 0.01$, y hecha con la PSF correcta).

Las tres perturbaciones empeoran el resultado. La que más afecta es la corrida (+17 % de RMSE), mientras que la ancha y la angosta suben 3.8 % y 3.0 %. Con la PSF corrida la imagen se ve igual de nítida, pero queda movida 1 píxel a la izquierda: si la devuelvo 1 píxel, la diferencia con la restauración correcta es solo 5×10^{-3}. Correr la PSF solo agrega una fase lineal a H ($|H|$ no cambia): cambia la posición y no la nitidez. En el mapa de error esto se ve como bordes con el doble de intensidad.

Con la PSF ancha se deconvoluciona de más y aparecen halos paralelos a los bordes, sobre todo en el abrigo y el trípode. Con la angosta se deconvoluciona de menos y queda un poco más borrosa que la correcta. Las dos tienen un error relativo parecido y casi el mismo RMSE, pero el tipo de artefacto es distinto.

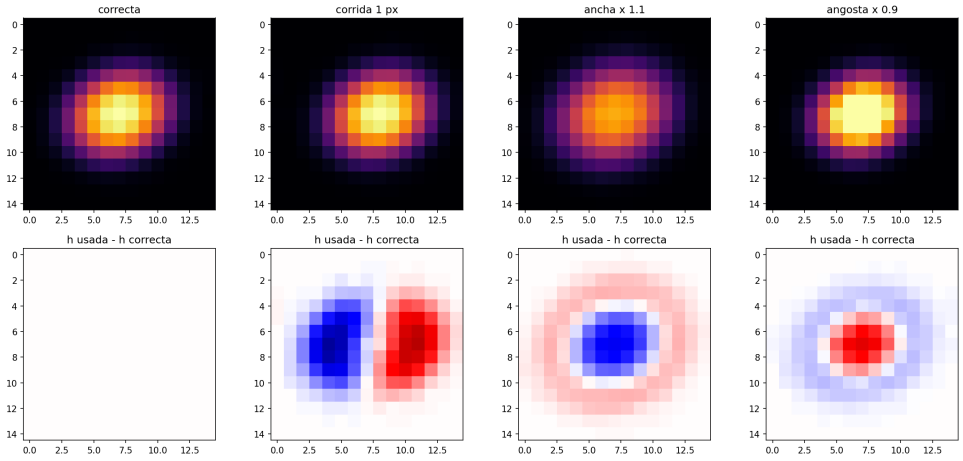

Figura 2.9: PSF usadas para restaurar (arriba) y diferencia con la correcta (abajo, escala ± 0.01).

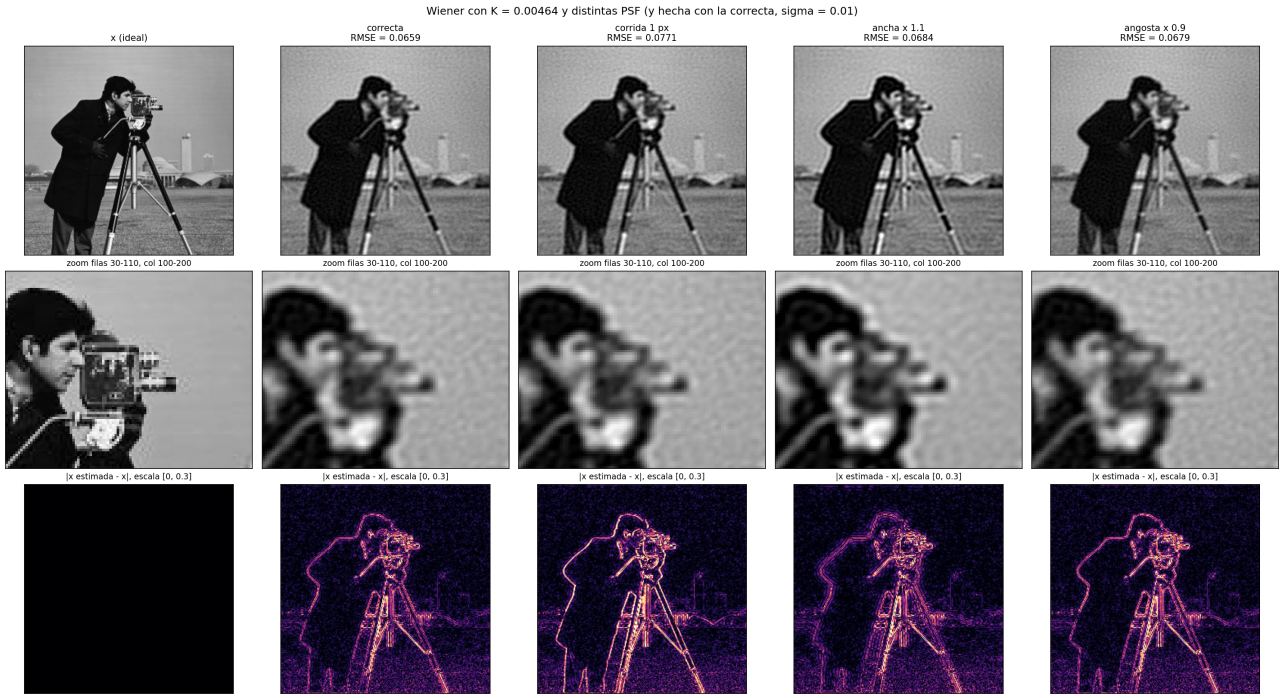

Figura 2.10: Wiener con $K = 4.64 \times 10^{-3}$ y las cuatro PSF. Filas: imagen completa, zoom (filas 30–110, columnas 100–200) y error $|\hat{x} - x|$ con escala $[0, 0.3]$.

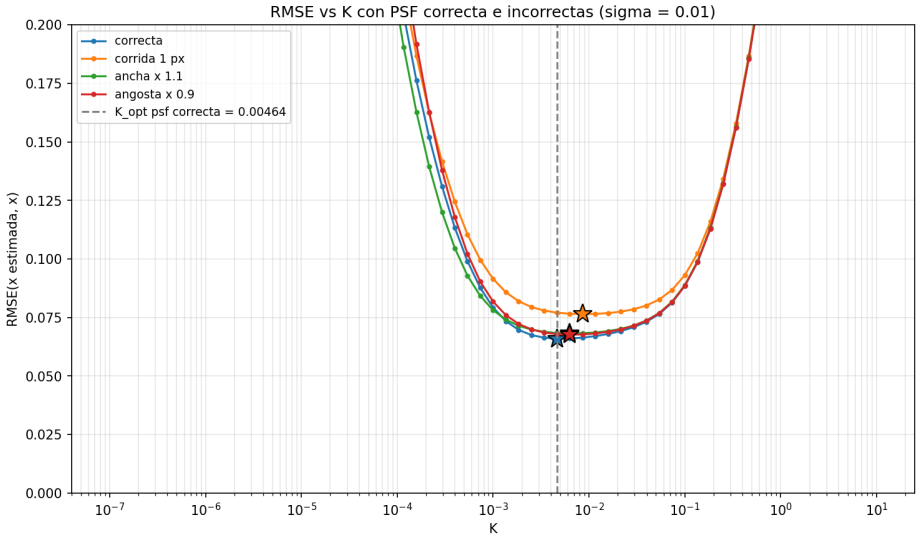

Figura 2.11: RMSE vs. K con la PSF correcta y las incorrectas. La línea punteada es el K_{opt} de la PSF correcta.

Con cualquier PSF incorrecta el K óptimo sube (6.3 y 8.6×10^{-3}): el error del modelo se comporta como ruido extra y conviene regularizar más. Pero volver a optimizar K casi no arregla nada (0.0771 a 0.0765), porque el problema está en la PSF y no en la regularización.

2.4. Tikhonov iterativo: λ, τ y criterio de detención

Antes de los barridos revisé la implementación. Con a y b aleatorias, $\langle Ha, b\rangle$ y $\langle a, H^T b\rangle$ difieren en 1.4×10^{-15} (relativo), entonces H^T es de verdad el adjunto de H. En una corrida de prueba con $\sigma = 0.01$, $\lambda = 4.64 \times 10^{-3}$, $\tau = 1$ y tol $= 10^{-6}$, el método se detuvo por el criterio en la iteración 1072, el costo bajó en todas las iteraciones (de 55.08 a 44.17) y el RMSE quedó en 0.0659, igual que Wiener con el mismo K.

Para el barrido usé $\sigma = 0.01$. Probé $\lambda = K_{\text{opt}}/10$, K_{opt} y $K_{\text{opt}} \times 10$ (4.64×10^{-4}, 4.64×10^{-3} y 4.64×10^{-2}), es decir, un orden de magnitud a cada lado del óptimo de Wiener, y además $\lambda = 0$. Para τ, con $\lambda = K_{\text{opt}}$ el límite de la Ecuación (2.4) es 1.991, y probé 0.5 (lento), 1.0 (seguro), 1.9 (cerca del límite) y 2.05 (pasado el límite). Para no hacer las 16 combinaciones, barrí τ con $\lambda = K_{\text{opt}}$ y λ con $\tau = 1$.

Usé un máximo de 3000 iteraciones. Con $\lambda = K_{\text{opt}}$ y $\tau = 1$ la frecuencia más lenta se reduce e veces cada $1/(\tau\lambda) \approx 215$ iteraciones, por lo que 3000 alcanzan de sobra; con $K_{\text{opt}}/10$ (unas 2150 por cada e) o $\lambda = 0$ ya se espera que no alcance. Corrí todo con tol $= 10^{-7}$ y después, con la lista de cambios relativos guardada, revisé en qué iteración se habría detenido con cada tolerancia.

	τ / λ	Iteraciones	Se detiene por	RMSE final	RMSE mínimo (iteración)
$\lambda = 4.64 \times 10^{-3}$	$\tau = 0.5$	2815	tol	0.0659	0.0658 (630)
	$\tau = 1.0$	1551	tol	0.0659	0.0658 (315)
	$\tau = 1.9$	886	tol	0.0659	0.0658 (165)
	$\tau = 2.05$	3000	máx. iteraciones	4.5×10^{72}	0.0679 (26)
$\tau = 1$	$\lambda = 0$	3000	máx. iteraciones	0.1403	0.0661 (172)
	$\lambda = 4.64 \times 10^{-4}$	3000	máx. iteraciones	0.0969	0.0660 (185)
	$\lambda = 4.64 \times 10^{-3}$	1551	tol	0.0659	0.0658 (315)
	$\lambda = 4.64 \times 10^{-2}$	194	tol	0.0747	0.0747 (194)

Tabla 2.4: Barridos de τ y de λ ($\sigma = 0.01$, máximo 3000 iteraciones, tol $= 10^{-7}$).

Convergencia lenta e inestabilidad (τ). Con $\tau < 2/(1+\lambda)$ las tres corridas llegan al mismo resultado (costo 44.17, RMSE 0.0659); solo cambia la velocidad, y el número de iteraciones va más o menos como $1/\tau$ (2815 con 0.5 y 886

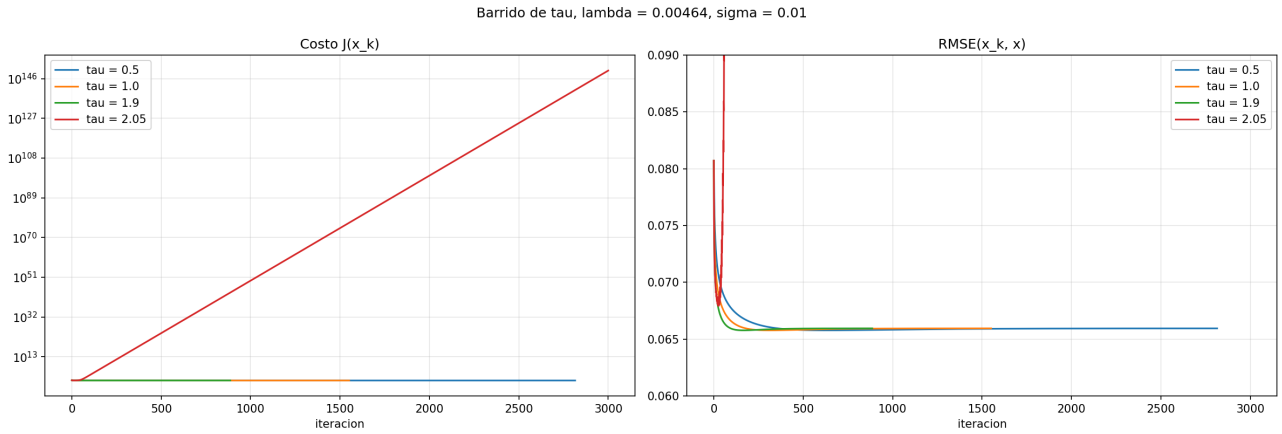

Figura 2.12: Costo y RMSE por iteración para distintos τ, con $\lambda = 4.64 \times 10^{-3}$ y $\sigma = 0.01$. El eje del RMSE está cortado en $[0.06, 0.09]$.

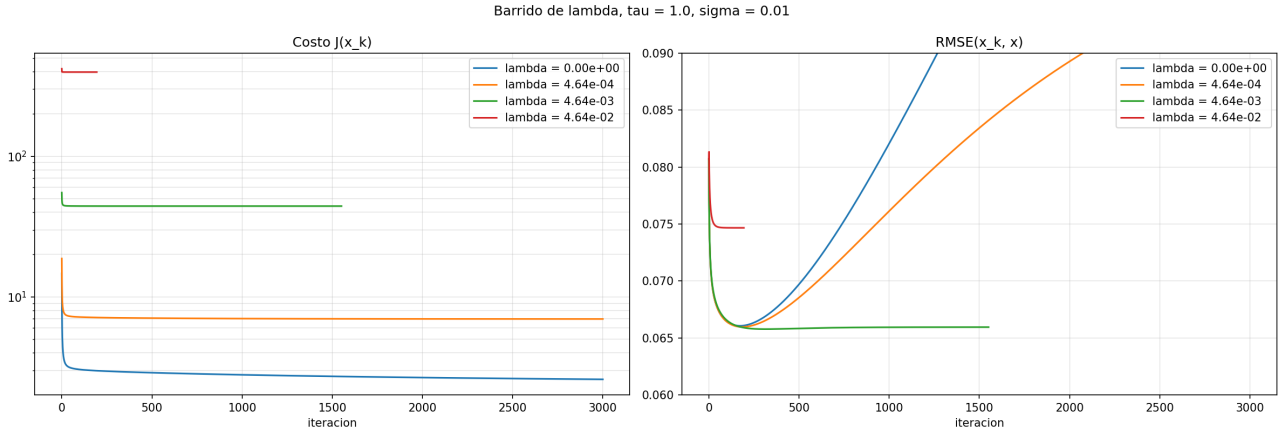

Figura 2.13: Costo y RMSE por iteración para distintos λ, con $\tau = 1$ y $\sigma = 0.01$.

con 1.9). Con $\tau = 2.05$ el método es inestable. En las primeras 26 iteraciones el RMSE baja, porque las frecuencias medias todavía convergen, pero la frecuencia $(0, 0)$, donde $|H| = 1$, tiene factor $|1 - 2.05(1 + \lambda)| = 1.0595$ y crece en cada iteración. El costo sube 1.12 veces por iteración ($= 1.0595^2$) y llega a 6.6×10^{149}. No llega a infinito en 3000 iteraciones y se detiene por el máximo y no por el corte de costo no finito.

Semi-convergencia, convergencia lenta y sobre-regularización (λ). Con $\lambda = 0$ el RMSE baja hasta 0.0661 en la iteración 172 y después sube sin parar (0.140 en la 3000). Es la semi-convergencia de la clase 17 [8]: primero se recuperan las frecuencias donde H es grande y después se empieza a amplificar el ruido donde H es chico, acercándose a Y/H. Con $\lambda = K_{\text{opt}}/10$ pasa algo parecido (mínimo en la 185, 0.0969 al final) y no cumple el criterio en 3000 iteraciones: converge lento hacia su propia solución, que tiene poca regularización. Con $\lambda = K_{\text{opt}}$ converge a 0.0659, igual que Wiener. Con $\lambda = K_{\text{opt}} \times 10$ converge rápido (194 iteraciones, $1/(\tau\lambda) \approx 22$), pero hacia una imagen sobre-regularizada, más suave y más oscura (0.0747). El costo final no sirve para comparar entre λ, porque cada λ es una función J distinta.

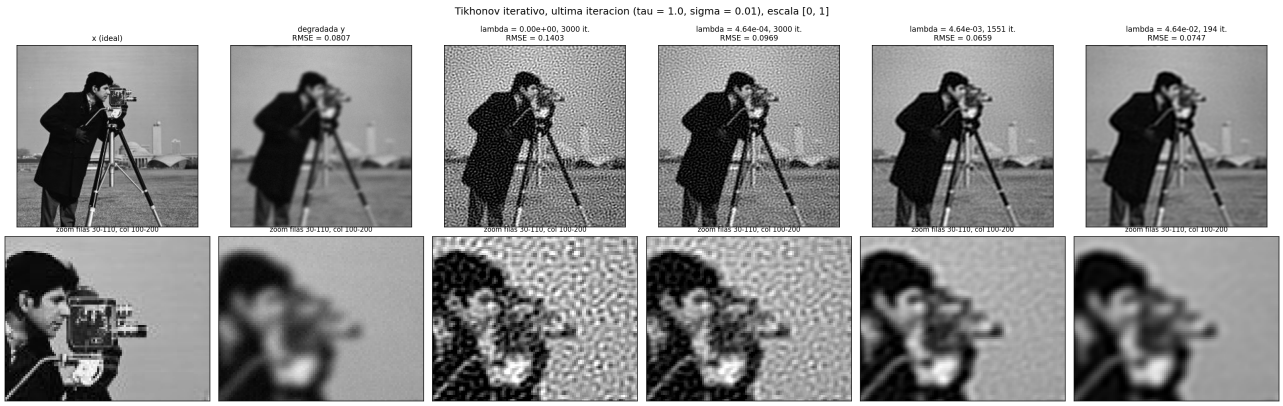

Figura 2.14: Última iteración de Tikhonov para cada λ ($\tau = 1$, $\sigma = 0.01$), con zoom. Escala $[0, 1]$.

tol	$\lambda = 0$	$\lambda = 4.64 \times 10^{-4}$	$\lambda = 4.64 \times 10^{-3}$	$\lambda = 4.64 \times 10^{-2}$
10^{-3}	22 it., 0.0692	21 it., 0.0693	20 it., 0.0696	12 it., 0.0760
10^{-4}	833 it., 0.0774	442 it., 0.0678	161 it., 0.0660	49 it., 0.0747
10^{-5}	no se cumple	no se cumple	598 it., 0.0659	97 it., 0.0747
10^{-6}	no se cumple	no se cumple	1072 it., 0.0659	146 it., 0.0747
10^{-7}	no se cumple	no se cumple	1551 it., 0.0659	194 it., 0.0747

Tabla 2.5: Iteración en que se detendría cada tolerancia y RMSE en ese punto ($\tau = 1$, máximo 3000 iteraciones).

Efecto del criterio de detención. Con $\lambda = K_{\text{opt}}$, desde tol $= 10^{-5}$ el RMSE ya no cambia y achicar la tolerancia solo agrega iteraciones (598 a 1551). Con 10^{-3} se detiene a las $\sim$20 iteraciones sin haber convergido. Con $\lambda = 0$ o λ chico pasa al revés: una tolerancia grande detiene antes y da mejor RMSE que dejarlo correr, porque detenerse temprano funciona como regularización. Con tolerancias chicas el criterio nunca se cumple y el resultado empeora.

2.5. Comparación con la solución cerrada

La solución cerrada de Tikhonov con λ es `wiener(y, H, λ)` (ver la pregunta guiada 6). Para compararla con la iterativa usé $\tau = 1.9$, el más rápido de los estables, tol $= 10^{-9}$ para acercarme lo más posible y un máximo de 3000 iteraciones, con los dos niveles de ruido y su K_{opt}. Para medir la distancia a la solución cerrada en cada iteración le pasé la solución cerrada como imagen de referencia a `tikhonov_gd`, así el RMSE que guarda es $\text{RMSE}(x^k, \hat{x}_{\text{cerrada}})$. La "mejor iterativa" es la iteración con menor RMSE contra x; para obtener esa imagen volví a correr con el máximo de iteraciones igual a esa iteración.

$\lambda = K$. La iterativa converge a la solución cerrada: quedan a 5.7×10^{-8} con $\sigma = 0.01$ y a 2.4×10^{-10} con $\sigma = 0.05$: son la misma solución. En la Figura 2.15 la distancia baja como una recta en escala logarítmica, con la pendiente de

σ	λ	Cerrada	Iterativa convergida	Iterativa vs. cerrada	Mejor iterativa
0.01	$K = 4.64 \times 10^{-3}$	0.06595	0.06595 (1393 it.)	5.7×10^{-8}	0.06578 (it. 165)
0.01	0	42.28	0.198 (3000 it.)	–	0.06606 (it. 91)
0.05	$K = 2.93 \times 10^{-2}$	0.07861	0.07861 (393 it.)	2.4×10^{-10}	0.07778 (it. 28)
0.05	0	211.4	0.963 (3000 it.)	–	0.08913 (it. 4)

Tabla 2.6: RMSE contra x de la solución cerrada y de la iterativa ($\tau = 1.9$, tol $= 10^{-9}$). "Iterativa vs. cerrada" es el RMSE entre las dos (diferencia máxima 2.5×10^{-7} y 2.9×10^{-10}). Con $\lambda = 0$ la iterativa no converge y se muestra la iteración 3000.

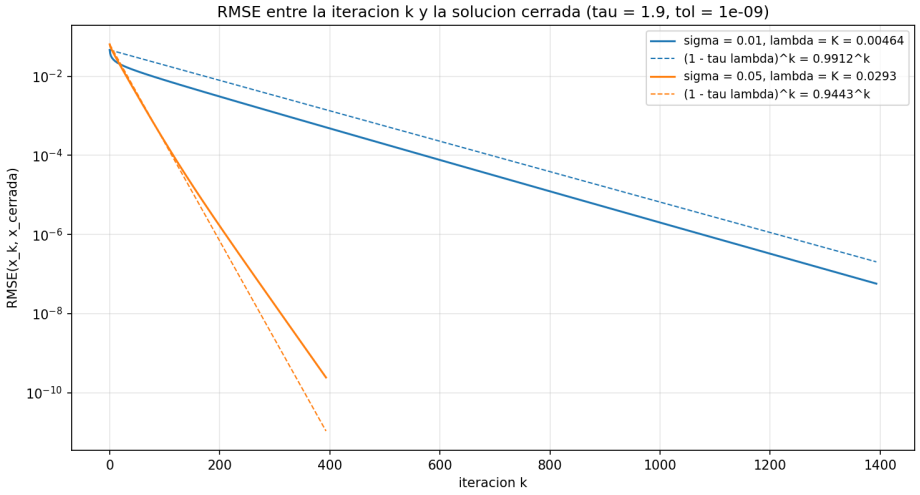

Figura 2.15: RMSE entre la iteración k y la solución cerrada con $\lambda = K_{\text{opt}}$. En línea punteada, $(1 - \tau\lambda)^k$.

la frecuencia más lenta, que se multiplica por $\max\left(1-\tau\lambda,\ |1-\tau(1+\lambda)|\right)$ en cada iteración. Con $\sigma = 0.01$ ese factor es $1 - \tau\lambda = 0.9912$ (frecuencias con $H \approx 0$), y por eso necesita 1393 iteraciones. Con $\sigma = 0.05$, $1 - \tau\lambda = 0.9443$, pero con $\tau = 1.9$ la frecuencia $(0, 0)$ tiene $|1 - 1.9 \cdot 1.0293| = 0.9557$, que es más lenta: la pendiente medida al final es 0.9556. Con τ cerca del límite la frecuencia cero oscila y pasa a ser la más lenta.

La mejor iteración es un poco mejor que la cerrada (0.06578 contra 0.06595 y 0.07778 contra 0.07861). Como dice la clase 17, la solución iterativa detenida antes puede ser mejor que la cerrada, porque las frecuencias con H chico todavía no llegan a su valor final y eso agrega un poco de regularización.

$\lambda = 0$. La solución cerrada es el filtro inverso, con RMSE 42.3 y 211, que no sirve. La iterativa nunca converge, pero detenida en el momento justo da 0.0661 (iteración 91, $\sigma = 0.01$) y 0.0891 (iteración 4, $\sigma = 0.05$). Con $\sigma = 0.01$ queda casi igual que Wiener; con $\sigma = 0.05$ hay que detenerla muy temprano y queda peor que Wiener (0.0891 contra 0.0786) y con más ruido. Si se deja correr empeora (0.198 y 0.963 en la iteración 3000). Además, la iteración óptima depende del ruido (91 contra 4), y en la práctica no se conoce x para elegirla. Por eso un λ explícito es más fácil de usar que adivinar cuándo detenerse.

En la Figura 2.16, con $\sigma = 0.01$ las tres reconstrucciones se ven casi iguales: recuperan el borde del pelo, la cara y la cámara, y en el cielo queda una textura suave de manchas. Con $\sigma = 0.05$, Wiener y Tikhonov con $\lambda = K_{\text{opt}}$ se ven parecidos y más suaves, mientras que Tikhonov con $\lambda = 0$ detenido en la iteración 4 casi no filtró el ruido y se ve granulado y menos nítido.

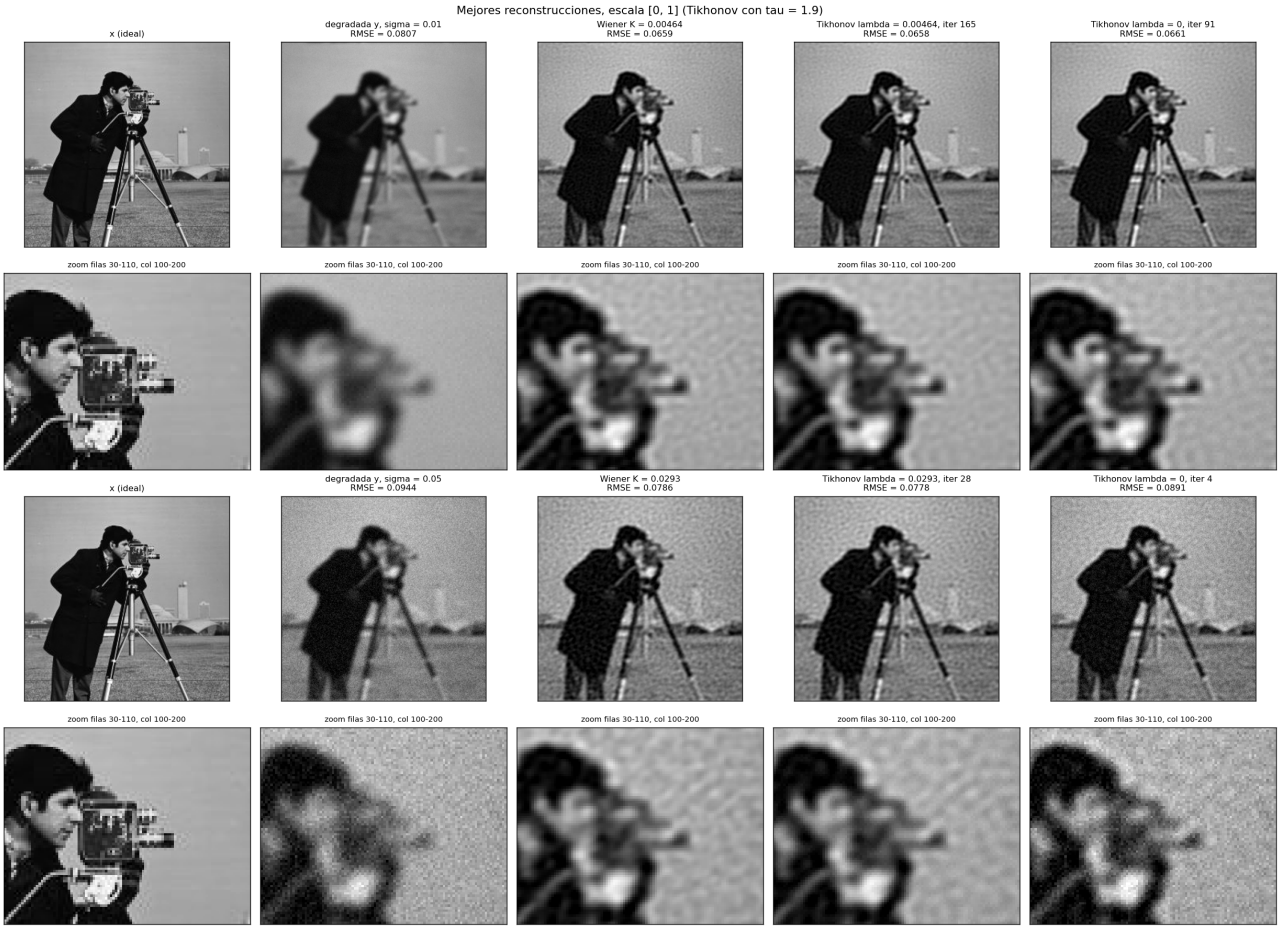

Figura 2.16: Mejores reconstrucciones. Filas 1–2: $\sigma = 0.01$. Filas 3–4: $\sigma = 0.05$. Columnas: ideal, degradada, Wiener con K_{opt}, mejor iteración de Tikhonov con $\lambda = K_{\text{opt}}$ y mejor iteración con $\lambda = 0$ ($\tau = 1.9$). Zoom: filas 30–110, columnas 100–200. Escala $[0, 1]$.

2.6. Respuestas a las preguntas guiadas

1 Por qué la inversión directa Y/H puede ser inestable

Con ruido, $Y = HX + N$, así que $Y/H = X + N/H$. H es un pasa bajos: en este caso $|H| < 0.01$ en el 78 % de las frecuencias y llega a 2×10^{-6}, mientras que el ruido blanco tiene la misma potencia en todas las frecuencias. En esas frecuencias N/H se vuelve enorme: la amplificación máxima es $1/\min|H| = 5 \times 10^5$. En la Figura 2.17 se ve que $|N|$ es plano y que $|N/H|$ copia la forma de $1/|H|$, con valores de 10^5 a 10^6 en los ceros de H. Por eso el filtro inverso da RMSE 3.6×10^{-13} sin ruido, pero 42.3 con $\sigma = 0.01$: el error es justamente el término N/H.

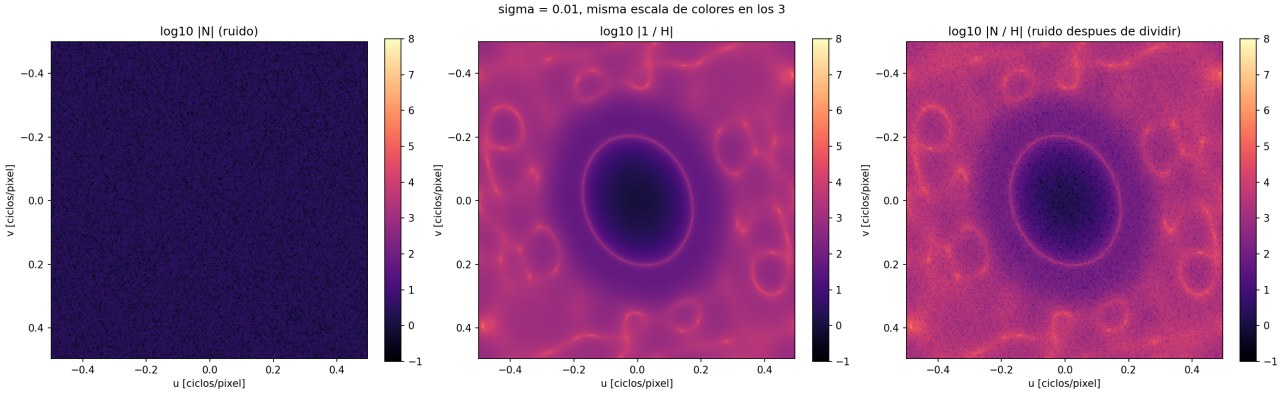

Figura 2.17: $\log_{10}|N|$, $\log_{10}|1/H|$ y $\log_{10}|N/H|$ con $\sigma = 0.01$, con la misma escala de colores.

El término K evita esa amplificación. La ganancia de Wiener es $|W| = |H|/(|H|^2 + K)$. Donde $|H|^2 \gg K$, $|W| \approx 1/|H|$, igual que el inverso (frecuencias bajas); donde $|H|^2 \ll K$, $|W| \approx |H|/K \to 0$ y la frecuencia se apaga en vez de amplificarse [7]. Derivando respecto de $|H|$, el máximo está en $|H| = \sqrt{K}$ y vale $1/(2\sqrt{K})$; con $K > 0$ ninguna frecuencia se multiplica por más que eso. Con la H de la PSF, el máximo de $|W|$ en la grilla da exactamente 50.00, 15.81 y 5.00 para $K = 10^{-4}$, 10^{-3} y 10^{-2} (Figura 2.18).

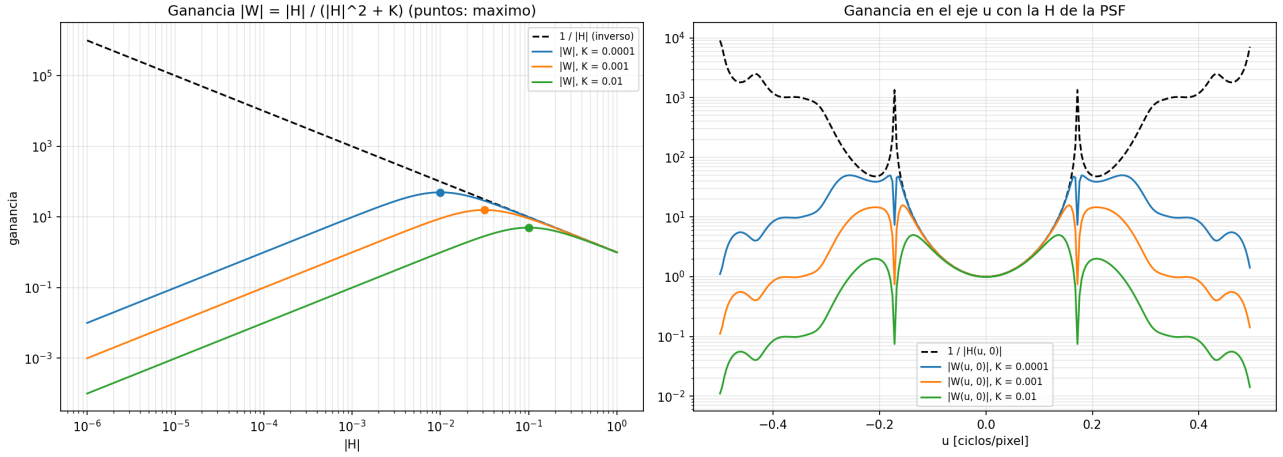

Figura 2.18: Izquierda: ganancia $|W|$ en función de $|H|$ para tres valores de prueba de K, comparada con $1/|H|$; los puntos marcan el máximo $1/(2\sqrt{K})$. Derecha: lo mismo en el eje u con la H de la PSF.

2 De la PSF a $H(u, v)$

Se hace en `psf_a_otf` (`src/pregunta2.py`). El ajuste de tamaño está en la línea 27: la PSF de 15×15 se copia en la esquina de una matriz de ceros del tamaño de la imagen (256×256). El centrado está en la línea 30: `np.roll` de $(-7, -7)$ mueve el centro $(7, 7)$ de la PSF al píxel $(0, 0)$, que es el origen para la FFT. Después se calcula `fft2` sin `fftshift`.

Si se omite el `np.roll`, $|H|$ es el mismo pero cambia la fase, y la imagen sale movida 7 píxeles hacia abajo y a la derecha: es exactamente la correcta desplazada (diferencia 5×10^{-16}), pero la diferencia con la correcta llega a 0.76 en los bordes de los objetos (Figura 2.19). Con un error de solo 1 píxel el RMSE de Wiener ya sube un 17 % (Tabla 2.3). Si el tamaño no coincidiera con el de la imagen, H y Y no tendrían la misma grilla de frecuencias y no se podrían multiplicar; y si se aplicara `fftshift` a H pero no a Y, cada frecuencia quedaría multiplicada por el valor de otra.

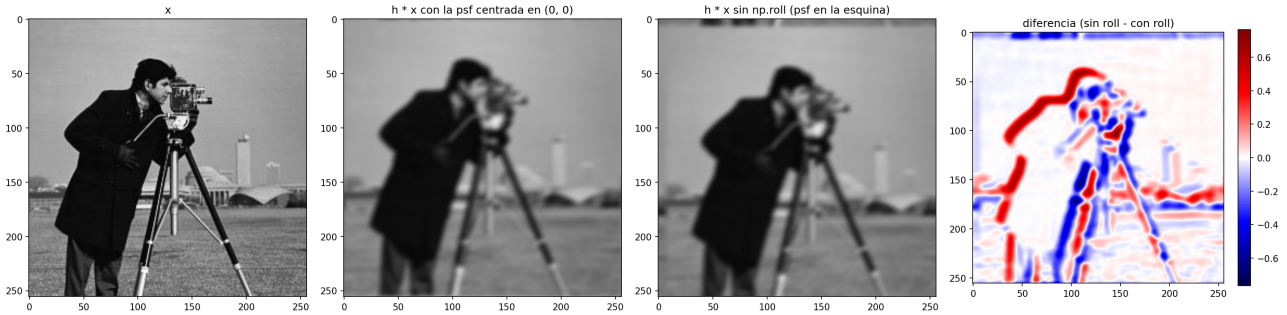

Figura 2.19: x, $h * x$ con la PSF centrada en $(0, 0)$, $h * x$ sin `np.roll` y la diferencia entre las dos.

3 Gradiente de Tikhonov

El gradiente es $\nabla J(x) = H^T(Hx - y) + \lambda x$. En `tikhonov_gd`, el término de fidelidad es `grad_fidelidad = aplicar_Ht(residuo, H)`, con `residuo = aplicar_H(x, H) - y` (línea 154), y el término de regularización es `grad_regularizacion = lam * x` (línea 155). Los dos se suman en `gradiente` y la actualización es `x_nuevo = x - tau * gradiente`, que es la Ecuación (2.2).

4 Cómo se implementa H^T

`aplicar_Ht` multiplica por H^* en Fourier. Como H es la DFT de la PSF, H^* es la DFT de la PSF girada en 180°: H^T es la correlación con h, o la convolución con la PSF girada. Lo comprobé de dos formas: `psf_a_otf` de la PSF girada da H^* (diferencia 2.7×10^{-16}) y la prueba del adjunto $\langle Ha, b\rangle = \langle a, H^T b\rangle$ da un error relativo de 1.4×10^{-15}.

Si la PSF fuera simétrica, H sería real y $H^T = H$. La PSF entregada es casi simétrica (0.78 %), por eso $H^T y$ y Hy difieren solo 8.5×10^{-4}. Para ver qué cambia con una PSF no simétrica, probé el descenso de gradiente usando H en lugar de H^T, con la PSF original y con la PSF corrida 1 píxel como si fuera la real (asimetría de 52.6 %), haciendo y con esa misma PSF. Con H en vez de H^*, cada frecuencia se multiplica por $1 - \tau(H^2 + \lambda)$ y el punto fijo pasa a ser $HY/(H^2 + \lambda)$.

PSF	Asimetría	RMSE con H^T	RMSE con H	El costo sube	$\max \lvert x_H - x_{H^T} \rvert$
Original	0.78 %	0.0659	0.0659	0 veces	0.006
Corrida 1 píxel	52.6 %	0.0660	0.0700	211 veces	0.143

Tabla 2.7: Descenso de gradiente con H^T y con H en su lugar ($\sigma = 0.01$, $\lambda = 4.64 \times 10^{-3}$, $\tau = 1$, 1000 iteraciones).

Con la PSF original casi no se nota, porque H es casi real. Con la PSF corrida, H tiene fase y la parte real de H^2 llega a ser negativa (mínimo -0.0015). En este caso no diverge, porque esas frecuencias tienen $|H|$ chico y ningún factor pasa de 1 (el máximo es 0.9969), pero converge a otra solución: RMSE 0.070 contra 0.066, igual al del punto fijo $HY/(H^2 + \lambda)$ (0.07001). Además el costo sube 211 veces, porque con H la dirección usada ya no es el gradiente de J y no hay garantía de que baje (Figura 2.20). Si la parte real de H^2 fuera más negativa en frecuencias donde $|H|$ es grande, el factor pasaría de 1 y el método divergería.

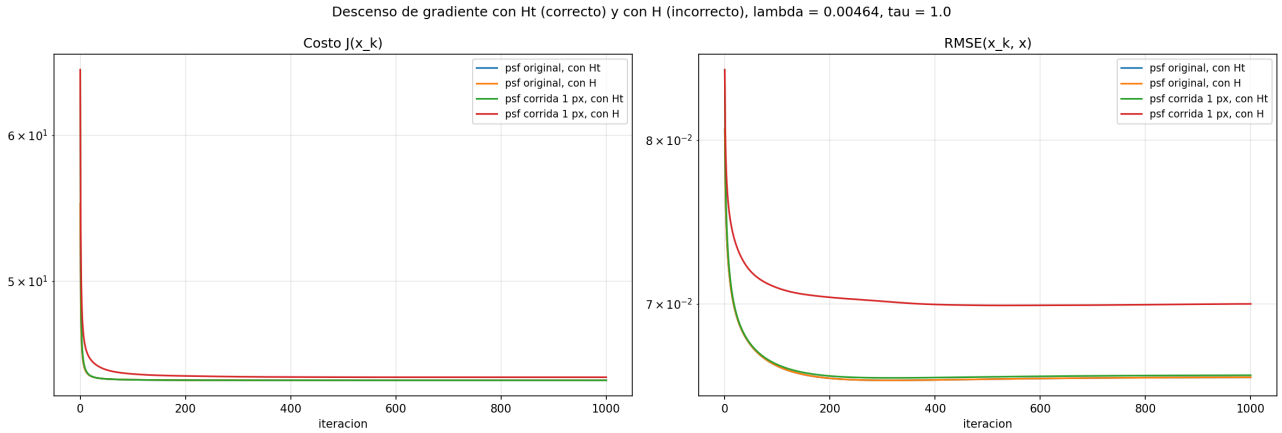

Figura 2.20: Costo y RMSE por iteración usando H^T (correcto) y H (incorrecto), con la PSF original y la corrida 1 píxel.

5 Criterio de detención

Calculo el cambio relativo de la Ecuación (2.3), $\|x^{k+1} - x^k\|_2 / \|x^k\|_2$, entre la iteración nueva y la anterior, dentro del `for` de `tikhonov_gd`, justo después de calcular `x_nuevo` (línea 164). La decisión se toma en la línea 179: `if cambio < tol: break`. Si nunca se cumple, se detiene al llegar a `max_iter`, y antes de eso también se detiene si el costo deja de ser finito. El efecto de la tolerancia está en la Tabla 2.5: con $\lambda = K_{\text{opt}}$, tol $\leq 10^{-5}$ ya da el resultado final, y con $\lambda = 0$ una tolerancia grande detiene antes y da mejor RMSE. Hay que tener en cuenta que el criterio mide cuánto cambia x y no qué tan lejos está de la solución, y por eso depende también de τ: con $\tau = 0.5$ hicieron falta 2815 iteraciones para el mismo tol que con $\tau = 1.9$ se cumplió en 886.

6 Solución cerrada de Tikhonov y Wiener simplificado

Igualando el gradiente a cero [8]:

$$H^T(H\hat{x} - y) + \lambda\hat{x} = 0 \implies (H^TH + \lambda I)\,\hat{x} = H^Ty \implies \hat{x} = (H^TH + \lambda I)^{-1}H^Ty. \tag{2.5}$$

Con convolución circular la DFT diagonaliza H: $Hx \leftrightarrow H(u,v)X(u,v)$, $H^Ty \leftrightarrow H^*(u,v)Y(u,v)$ y $H^THx \leftrightarrow |H(u,v)|^2X(u,v)$. Entonces la ecuación queda separada por frecuencia, $(|H|^2 + \lambda)\hat{X} = H^*Y$, y

$$\hat{X}(u,v) = \frac{H^*(u,v)\,Y(u,v)}{|H(u,v)|^2 + \lambda}, \tag{2.6}$$

que es el Wiener simplificado de la Ecuación (2.1) con $K = \lambda$.

Numéricamente coinciden hasta 5.7×10^{-8} y 2.4×10^{-10} (Tabla 2.6). Lo que impide una coincidencia exacta es, primero, que la iterativa se detiene después de un número finito de iteraciones: el error baja como (factor)^k y depende de τ, λ, tol y `max_iter`. Por ejemplo, con $\tau = 1$ y tol $= 10^{-6}$ (la corrida de prueba, 1072 iteraciones) la diferencia con la cerrada es 1.1×10^{-4}. También influyen los errores de redondeo de las FFT, que se acumulan en miles de iteraciones, y el mal condicionamiento cuando λ es chico o 0 ($|H|^2$ llega a 4×10^{-12}): ahí la iterativa no llega a la cerrada en un número razonable de iteraciones. Por último, H y H^T tienen que usar el mismo modelo de borde que la fórmula de Fourier. Si la iterativa usara otro borde (padding de ceros o reflejo), ya no sería la misma matriz diagonalizada por la DFT y las soluciones no coincidirían.

Código y material de referencia

Implementación: `src/pregunta2.py`. Experimentos, en el orden en que se hicieron: `test_p2_otf`, `test_p2_degradacion`, `test_p2_wiener_inversa`, `test_p2_wiener_barrido`, `test_p2_psf_incorrecta`, `test_p2_tikhonov_check`, `test_p2_tikhonov_barrido`, `test_p2_comparacion` y `test_p2_preguntas`, en `experiments`. El README indica cómo ejecutarlos.

Referencias

[1] Cápsulas del curso. *fundamentos-img*, material de apoyo de las semanas 8 (Transformada de Fourier), 9 (Filtros en frecuencia y PSF) y 10 (Deconvolución)

[2] *Clase 2 – Visión humana y sensores*

[3] *Clase 11 – Frecuencia*

[4] *Clase 14 – Filtros pasa altos y pasa banda*

[5] *Clase 7 – Operatoria de imágenes e interpolación*

[6] *Clase 15 – PSF*

[7] *Clase 16 – Deconvoluciones*

[8] *Clase 17 – Bayes y regularización*